Source-Guided Roadmap for Reimplementing and Machine-Checking the HAETAE Provable-Security Proof

Technical report for the HAETAE EasyCrypt proof surface

May 2026

Abstract

This report is a repository-guided roadmap for reimplementing the HAETAE provable-security proof surface. It is written for a reader who has not seen the repository before and wants to understand the pen-and-paper proof boundary, the EasyCrypt file architecture, and the verification workflow. It is not, by itself, a literal replacement for the `.ec` source files: several blueprint snippets are schematic, and a faithful machine-checked reimplementation still requires the existing EasyCrypt files, generated declaration inventory, and EasyCrypt libraries as companion artifacts. A claim of report-only machine-checkable reproduction should therefore be rejected unless this roadmap is expanded into a complete source-and-proof archive. The report identifies the exact theorem being checked, separates checked facts from external assumptions, gives the source-file dependency order, lists the mathematical object families that must be introduced, and records the verification commands that certify the proof gate. The document-only reproducibility question has a negative answer: the report can explain the checked formal proof boundary, but it cannot by itself establish HAETAE algorithm security or reproduce the EasyCrypt verification. The checked theorem is a conditional random-oracle-model statement for the formal EasyCrypt model. It is not a standard-model SHAKE theorem, not an implementation-correctness theorem, and not a non-vacuous concrete security instantiation until the currently oversized rejection-sampling antecedent is repaired. When full `.ec` proof scripts are provided as companion artifacts, this report also states the documentation standard needed for a mathematician or cryptologist to understand the mathematical content of those scripts without relying on unstated repository lore. This version records that standard and an initial coverage ledger; it should not be described as complete script-companion documentation until every required coverage row is discharged.

Contents

Chapter 1

Scope and End Product

1.1 What the reader should be able to reproduce

After following this report together with the companion EasyCrypt sources, a reader should be able to recreate a proof surface with the following properties.

1. The EasyCrypt files compile as a manifest-driven proof gate.
2. The generic random-oracle signature game is instantiated by a formal HAETAE scheme module.
3. Correctness is proved for the formal model.
4. The EUF-CMA game is related to a sequence of simulated, transcript, NMA, sampling, and extraction games.
5. The final checked security implication has the same dependency boundary as the current artifact: it is conditional on explicit hop/reduction premises and abstract MLWE/Module-SIS hardness bounds.
6. The counted-ROM loss ledger is normalized to the concrete expression

$$\frac{37}{2^{58}}$$

   for the fixed wrapper constants $q_H = q_S = 2$.
7. The proof explicitly records the current vacuity issue: `rejection_sampling_loss_term` ≥ 1, and in fact the min-entropy part alone is 12, so a final premise that adds this term to a probability cannot be treated as a useful concrete-security premise.

1.2 Sufficiency status

This report should be read as a technical map and acceptance checklist, not as a complete standalone source of the proof. It is sufficient for understanding the shape and limits of the checked theorem, for navigating the proof files in the right order, and for checking that a reconstructed development has the expected manifest, theorem statement, and loss ledger. It is not sufficient, on its own, to type all EasyCrypt code from a blank file and recover the current proof.

The reason is structural rather than cosmetic. A machine-checked EasyCrypt development depends on exact declarations, module restriction sets, local helper lemmas, proof tactics, import paths, and library behavior. This report records the required proof architecture and representative source forms, but it does not inline all 22046 lines of checked EasyCrypt. Whenever a snippet contains ..., it is a blueprint placeholder, not compilable code. A reader attempting a literal reimplementation must replace every such placeholder by the corresponding complete definition or proof from the companion artifacts.

1.3 Document-only verdict

The question "Can a reader understand the provable security of the HAETAE digital signature algorithm and reproduce the machine-checked verification based solely on this technical document?" has the following answer.

> **No.** The document alone is enough to orient a reader to the checked formal-model theorem, but it is not enough to certify the HAETAE algorithm as a deployed digital signature scheme and it is not enough to recreate the machine-checked EasyCrypt artifact from a blank workspace.

There are three distinct claims, and only the weakest one is supported by the report alone.

1. **Understanding the checked theorem boundary: yes.** The report states the final theorem shape, the random-oracle setting, the fixed counted-ROM ledger, the declared hardness premises, and the rejection-sampling vacuity warning.
2. **Understanding full HAETAE algorithm security: no.** The report does not supply all bridges from the formal EasyCrypt model to the specification, SHAKE instantiation, concrete lattice estimates, implementation behavior, or side-channel properties.
3. **Reproducing machine-checked verification from the document alone: no.** The report omits literal EasyCrypt source, complete proof scripts, exact module restrictions, full helper-lemma bodies, and a full import/library contract. Those are exactly the objects that the checker consumes.

The strongest truthful claim is therefore conditional: with the companion source tree and libraries, the report guides a reader through the proof surface and the verification gate. Without those artifacts, it is an audit roadmap, not a standalone proof.

1.4 Can this document stand alone?

The criticism that motivated this revision is correct: the report, by itself, cannot be both a readable roadmap and a complete machine-checkable source archive. The following table states exactly what can and cannot be recovered from the document alone.

Goal	From this report alone?	Reason

Understand the final theorem boundary	Yes	The theorem shape, ROM ledger, declared hardness premises, and vacuity warning are stated explicitly.
Reconstruct the pen-and-paper proof outline	Partly	The hybrid order and loss accounting are given, but each step still has to be checked against the source lemmas that implement it.
Type a fresh EasyCrypt development from a blank directory	No	The report does not inline every declaration, helper lemma, proof script, import, and module restriction set.
Machine-check equivalence to the current artifact	No	Equivalence requires the companion source tree or an independently expanded document containing literal source and proof scripts.
Conclude end-to-end HAETAE implementation security	No	The checked theorem is formal-model, random-oracle-model, fixed-ledger, and conditional; byte-level, constant-time, SHAKE-instantiation, and concrete hardness claims are outside the checked result.

Consequently, any phrase such as "reproduce the machine-checked verification based solely on this report" must be read as false for the present artifact. The accurate claim is narrower: a reader can use the report to understand the proof boundary and to reproduce the verification when the report is paired with the listed source artifacts.

1.5 Formal-model understanding versus algorithm security

The phrase "the provable security of HAETAE" is ambiguous. In this report it means the checked EasyCrypt theorem about the formal HAETAE model, not a full security certification of every object that a cryptographer or implementer might call the HAETAE digital signature algorithm. A reader can learn the formal proof boundary from this document: the game being bounded, the random-oracle abstraction, the declared hardness games, the counted-ROM loss ledger, and the final vacuity warning. That is a real but narrower achievement than understanding an end-to-end algorithm-security theorem.

To turn formal-model understanding into algorithm-security understanding, the following bridges would also have to be supplied and checked or separately audited.

- A specification-to-model bridge showing that the EasyCrypt operations faithfully encode the HAETAE specification being claimed.
- A hash-instantiation argument explaining why the random-oracle theorem is applicable to the concrete SHAKE-based algorithm, or an explicit decision not to make that claim.
- Concrete parameter analysis for the MLWE and Module-SIS assumptions, including any reduction losses not internalized by the EasyCrypt theorem.
- Implementation-correctness and side-channel arguments for concrete code, if the claim is about deployed signing and verification programs.
- A repaired or reinterpreted rejection-sampling premise if the theorem is to be cited as a non-vacuous concrete bound.

Because those bridges are not all present as checked theorems, the document should be used to understand the formal verification surface, not as sole evidence that the deployed HAETAE algorithm has been proved secure.

1.6 Companion artifacts required

A faithful reimplementation therefore requires at least the following companion materials.

- The manifest files under `provable-security/easycrypt/`, or an equivalent complete source tree.
- The generic EasyCrypt libraries imported by those files.
- The `kyber-security` include path, unless every imported dependency from that path is audited and replaced.
- The declaration inventory generated by `dissertation/etc/count-easycrypt-declarations.sh`.
- The HAETAE specification PDF and model/specification audit material if the formal EasyCrypt model is to be interpreted as the February 2026 HAETAE specification.

Without those artifacts, the report can support a mathematical review of the proof boundary, but it cannot certify that a newly written EasyCrypt development is definitionally identical to the checked one.

1.7 Source-truth contract

A faithful reproduction must declare its source of truth before any theorem is accepted.

1. In *audit mode*, the source of truth is the existing `provable-security/easycrypt/` tree plus the manifest. The report is a guide for reading, checking, and rebuilding that tree.
2. In *literal standalone mode*, the source of truth would have to be a larger document that contains every active EasyCrypt file in full, every proof script, every module restriction set, and every imported helper dependency that affects type checking or proof checking.
3. In *independent reimplementation mode*, the new development may choose different local names or proof scripts, but it must provide a traceability matrix from each manifest file, declaration family, theorem, and loss term in this report to a checked EasyCrypt object in the new tree.

The current report intentionally supports audit mode and independent reimplementation mode with companion sources. It does not satisfy literal standalone mode. Treating it as if it did would erase exactly the facts that EasyCrypt checks: definitional equality, dependency resolution, tactic success, and module-separation restrictions.

1.8 Script companion mode

There is a fourth, weaker-than-standalone but stronger-than-roadmap use case: the `.ec` proof scripts are provided alongside this document, and the document is responsible for making their mathematical content understandable to mathematicians and cryptologists who have not previously studied the project. This is *script companion mode*.

Script companion mode does not mean that the prose replaces EasyCrypt. The scripts remain the machine-checkable artifact. It means that a reader should be able to open a script, look up every important declaration and proof block in this report, and understand what mathematical claim is being checked, why the claim is needed, which assumptions it uses, and what each proof step contributes. The reader may still need EasyCrypt to check the proof, but should not need unstated repository lore to understand the proof.

In script companion mode, the documentation must provide the following for each manifest file.

Documentation item	**Required content**
File role	The mathematical purpose of the file, its upstream dependencies, and the later files that rely on it.
Declaration dictionary	Every type, operation, module type, module, predicate, event, and distribution that is not standard EasyCrypt vocabulary, with a mathematical reading.
State interpretation	For every game or oracle module, the meaning of each state variable, log, counter, bad flag, and sampled value.
Restriction explanation	For every security-critical `declaremodule` restriction set, the information-flow reason for the exclusions.
Lemma ledger	A table of important lemmas and theorems giving the EasyCrypt name, the informal mathematical statement, dependencies, and the later result that consumes the lemma.
Proof-script commentary	A block-by-block explanation of the tactic script: which invariant is opened, which equivalence or probability rule is applied, which algebraic rewrite is used, and which side condition is delegated to `smt`.
Assumption boundary	A statement of whether a fact is checked locally, imported from a library, declared as a hardness interface, or left as an external cryptographic assumption.

This requirement is intentionally stronger than merely listing theorem names. For example, a line such as `byphoare` or `byequiv` must be explained by the game transformation it establishes; a call to `smt` must be associated with the arithmetic, set-theoretic, or losslessness side condition it discharges; and a module restriction list must be explained as an adversarial-separation condition rather than as syntax.

1.9 Per-lemma explanation template

Every nontrivial EasyCrypt lemma used by the final theorem should have an entry with the following fields.

1. **EasyCrypt anchor:** file name, lemma name, and whether the proof is local, imported, or a direct composition of earlier lemmas.

2. **Mathematical statement:** a prose or displayed-math rendering that does not require EasyCrypt syntax to parse.
3. **Objects involved:** the games, modules, distributions, events, state variables, and loss terms appearing in the statement.
4. **Assumptions:** module restrictions, losslessness facts, positivity facts, hardness premises, or previous hop lemmas consumed by the proof.
5. **Proof idea:** the mathematical argument before tactic details.
6. **Script reading:** an explanation of the proof script grouped into meaningful blocks, including what is proved by rewriting, by probabilistic program logic, by equivalence reasoning, and by SMT.
7. **Security role:** the place of the lemma in the global reduction: exact hop, bad-event bound, sampling-distance bound, extraction step, hardness adapter, or arithmetic normalization.
8. **Failure mode:** what would go wrong if the lemma were misstated, weakened, or used outside its hypotheses.

The goal is not to duplicate every tactic token in prose. The goal is that a mathematician or cryptologist can read the `.ec` script and this entry together and recover the proof argument without asking what an identifier, game state, or proof step was intended to mean.

1.10 Minimum EasyCrypt vocabulary for readers

When proof scripts are supplied, this report must also define the EasyCrypt vocabulary used by the scripts at the level needed to read the proof. At minimum, it must explain:

- `op`, `pred`, `type`, `moduletype`, `module`, and `declaremodule`;
- `proc`, program state, sampling, procedure calls, and return values;
- probability expressions `Pr[...@&m:...]`;
- Hoare, probabilistic Hoare, and equivalence judgments;
- `byphoare`, `byequiv`, `islossless`, `lossless`, `rewrite`, `case`, `move`, `apply`, and `smt`;
- the meaning of module restriction sets such as `-H,-A,-HAETAE`;
- the difference between a checked lemma, a declared module interface, and an external cryptographic assumption.

These explanations should be tied to the HAETAE scripts, not presented as a generic EasyCrypt tutorial. Each tactic or syntactic form should be introduced where it first matters for a HAETAE proof step.

1.11 Current script-companion coverage status

The criticism that this document still only states the script-companion standard is correct. The current report contains a useful roadmap and several mathematical readings, but it is not yet a complete commentary for every security-relevant block of the supplied `.ec` scripts. The table below is therefore part of the technical specification: it records what remains before the document can honestly claim that mathematicians and cryptologists can understand the scripts' proof content solely through this report plus the scripts themselves.

Coverage item	**Current status**	**Required to close**
File roles for manifest files	Partial	The file-by-file blueprint gives the main role of each file, but it must be cross-checked against the actual `.ec` scripts and expanded with upstream and downstream dependencies.
Declaration dictionary	Partial	Every nonstandard type, operation, predicate, distribution, module type, module, and event in the scripts must have a mathematical reading and script anchor.
State interpretation	Partial	Game modules, signing oracles, random-oracle modules, transcript simulators, and samplers need complete state-variable tables.
Restriction explanations	Partial	Representative restriction sets are shown, but every security-critical `declaremodule` restriction must be explained by its information-flow purpose.
Lemma ledger	Partial	The report lists important lemma names, but each nontrivial lemma used in the final theorem needs a full entry using the per-lemma template.
Proof-script commentary	Not complete	Tactic scripts must be grouped into mathematical proof blocks explaining `byphoare`, `byequiv`, rewrites, losslessness arguments, and `smt` side conditions.
EasyCrypt vocabulary tied to HAETAE	Partial	The report states the vocabulary that must be explained, but the explanations must be integrated at the first HAETAE proof point where each construct matters.
Assumption boundary	Partial	The major boundaries are recorded, but each imported library fact, declared hardness interface, and external cryptographic assumption must be labelled in the lemma ledger.

Until the rows above are complete, the report should be cited as a script-companion specification and partial reading guide, not as a completed script-companion document. The completion test is concrete: for every security-relevant declaration and proof block in the supplied `.ec` scripts, a reader must be able to find a nearby entry in this report explaining the mathematical statement, the proof idea, the tactic role, and the place of the block in the global security reduction.

1.12 Coverage ledger template

A complete script-companion version should maintain a ledger with one row per security-relevant script block. The ledger may be embedded in this report or generated as an appendix, but it must be reviewable as prose. Each row should have the following shape.

Field	Meaning
Script anchor	File name, line or lemma name, and whether the block is a definition, module, declaration, lemma, proof, or tactic sub-block.
Mathematical reading	The human mathematical object or claim represented by the script block.
Dependencies	Prior declarations, imported theories, assumptions, module restrictions, or lemmas needed to read the block.
Proof explanation	For proof blocks, the argument split into exact equivalence, probabilistic bound, invariant preservation, algebraic rewrite, losslessness proof, extraction step, or arithmetic normalization.
Checker role	What EasyCrypt verifies at this block: type checking, losslessness, Hoare judgment, equivalence judgment, probability inequality, or SMT side condition.
Security role	Why the block matters for the HAETAE proof: correctness, hybrid hop, random-oracle programming, transcript validity, rejection bound, hardness adapter, or final theorem composition.
Status	Complete only when the prose is sufficient for a cryptographer to read the script block without guessing the intended mathematics.

This ledger is the mechanism that turns the current roadmap into a genuine script companion. A build of the LaTeX document is not enough; the ledger must also be substantively complete against the supplied scripts.

1.13 Initial filled coverage entries

The entries below are the first concrete script-companion rows. They are included to demonstrate the required level of explanation and to prevent the coverage ledger from remaining a purely aspirational template. The status is "partial" unless the row covers every dependent declaration and proof block in the surrounding script region.

Script anchor	Mathematical reading	Current explanation and remaining work

`Sig_ROM.ec`, types and freshness operations	Generic signature-game vocabulary. The abstract types `pkey`, `skey`, `message`, `context`, `signature`, `ro_query`, and `ro_output` define the carrier sets of a random-oracle signature experiment. The predicates `fresh_msg` and `fresh_sig` express whether a forgery target was absent from the signing log.	The report explains the EUF-CMA/NMA game boundary and freshness condition. To close the row, add a declaration dictionary entry for every generic type, the signed-query variant used by SUF-CMA, and the exact list operations used in the freshness lemmas.
`Sig_ROM.ec`, `EUF_CMA` and `UF_NMA` modules	`EUF_CMA` is the adaptive chosen-message experiment with a signing oracle that records queried message-context pairs. `UF_NMA` is the no-message experiment in which the adversary only has random-oracle access.	The mathematical game notation table identifies both games. To close the row, add state-variable tables for `EUF_CMA.sk`, `EUF_CMA.queries`, and the local variables `pk`, `sk`, `m`, `ctx`, `sig`, and `ok`.
`Sig_ROM.ec`, `nma_as_cma_exact`	The NMA adversary adapter is distribution-preserving when embedded into the CMA game with an unused signing oracle: the two success probabilities are equal.	The proof is an EasyCrypt equivalence proof using `byequiv`, inlining the adapter and simulating matching calls to the adversary, scheme, and oracle. To close the row, add a block-by-block explanation of the `call`, `wp`, `sim`, and final `auto` steps.
`HAETAE_Reductions.ec`, hardness and loss operations	The operations `mlwe_hardness_term`, `module_sis_hardness_term`, and `bimodal_to_msis_reduction_loss_term` are abstract real-valued assumption/loss slots. The counted-ROM terms encode collision, prequery, and min-entropy losses for $q_H = q_S = 2$ and challenge support 2^{58}.	The report gives the counted-ROM arithmetic and the $37/2^{58}$ calculation. To close the row, add dictionary entries for every real-valued operation in the file and identify which are abstract assumptions, definitions, or checked arithmetic normalizations.

`HAETAE_Reductions.ec`, `haetae_euf_counted_rom_bound_groupedE`	This lemma rewrites the final counted-ROM bound into the sum of MLWE hardness, Module-SIS hardness, bimodal-to-MSIS loss, and counted random-oracle programming loss.	The proof is pure real algebra: unfold the grouped definitions and discharge the equality by the ring tactic. To close the row, add similar explanation entries for the nonnegativity, positivity, subunit, concrete-cardinality, and vacuity lemmas in the same file.
`HAETAE_HopGames.ec`, section `ROMInternalTranscriptROSigningAttemptExactHyperballPaperSimNMA`	This region relates an NMA game using a random-oracle signing attempt to one using the exact-hyperball sampler, while tracking rejection and sampler-prequery losses.	The report describes the layer as NMA and transcript game lifting. To close the row, add commentary for the paper-sample lifting lemma, the budgeted lifting lemmas, the clean-conditioned lifting lemma, and the bad prequery bound that is imported into the inequality chain.
`HAETAE_HopGames.ec`, `rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting`	The lemma decomposes the success event into a clean branch and a sampler prequery bad event. It bounds the clean branch by the exact-hyperball game plus rejection loss, and charges the bad branch to a counted-ROM prequery term.	The script uses union/subdistribution inequalities, a clean-lifting premise, and the lemma `budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound`. To close the row, explain each `ler_trans`, `Pr[mu_sub]`, `Pr[mu_or]`, `ler_add`, and final bad-event-bound application.
`HAETAE_Security.ec`, section `MechanizedROMTranscriptConstructedNMALift`	This section is the final composition lane. It progressively constructed NMA and EUF-CMA views by structural, public-simulation, paper-simulation, rejection-simulation, exact-hyperball, and random-oracle exact-hyperball NMA views before applying MLWE and bimodal/Module-SIS assumptions.	The report states the final theorem shape and representative restriction sets. To close the row, every lemma in this section must be entered in the ledger, including its predecessor theorem, hop premise, hardness premise, arithmetic bound, and whether the proof is a rewrite, transitivity step, or final composition.

HAETAE_Security.ec, declaremoduleH, A, Samp, Bmlwe, Bbimodal in the final composition section	The restrictions enforce adversarial separation: the random oracle and adversary cannot call hidden game internals, the sampler cannot inspect the adversary or oracle beyond its interface, and the hardness solvers remain abstract modules.	The report gives a representative restriction block and explains why restriction sets are proof obligations. To close the row, every excluded module name in the final block must be mapped to the hidden state or procedure it protects.

1.14 Script companion coverage for Sig ROM

The generic signature-game file is small enough that its script-companion coverage can be closed in this report. The file is an abstract theory: it does not know HAETAE-specific algebra, rejection sampling, or lattice assumptions. It defines the reusable game vocabulary that later files instantiate with the HAETAE scheme and random oracle.

Script item	**Mathematical reading**
pkey, skey, message, context, signature	Abstract carrier sets for public keys, secret keys, signed messages, auxiliary contexts, and signatures. No algebraic structure is assumed here.
ro_query, ro_output, dro_output	The input and output domains of the random oracle, together with a lossless output distribution used by the cloned full-random-oracle library.
query, signed_query	The signing-log domains. A query records a message-context pair; a signed_query records the same pair together with the signature returned by the signing oracle.
fresh_msg	EUF-CMA freshness: the message-context pair in the alleged forgery was not previously submitted to the signing oracle.
fresh_sig	Strong unforgeability freshness: the full message-context-signature triple was not previously returned by the signing oracle.
Oracle and POracle	Oracle exposes random-oracle initialization and query access; POracle exposes query access only. Adversaries receive POracle so they cannot reset the oracle.
Scheme(O)	The signature-scheme interface parameterized by public oracle access: key generation, signing, and verification.
CORR_ADV(O)	The correctness-game adversary, which chooses the message and context after seeing both public and secret keys.
Adversary(H,O)	The EUF-CMA adversary. Its procedure may call the random oracle and the signing oracle.
NMA_Adversary(H)	The no-message adversary. It may call the random oracle but has no signing oracle.
NMA_As_CMA	An adapter that lets an NMA adversary inhabit the CMA adversary interface while ignoring the signing oracle.

The game modules have the following state interpretation.

Game state or local	Meaning
`Correctness.main`	Initializes the random oracle, samples a key pair, asks the correctness adversary for a message and context, signs them, verifies the signature, and returns failure $\neg ok$. Thus a zero probability result is perfect correctness for the formal model.
`EUF_CMA.sk`	The secret key stored by the EUF-CMA game so the internal signing oracle can answer adversarial signing queries.
`EUF_CMA.queries`	The list of message-context pairs queried to the signing oracle. It is reset to the empty list before the adversary runs.
`EUF_CMA.O.sign`	The signing oracle: it signs with `sk`, logs (m, ctx), and returns the signature.
`EUF_CMA.main`	The adaptive unforgeability experiment. It accepts iff verification succeeds and `fresh_msg` holds for the forgery message-context pair.
`UF_NMA.main`	The no-message experiment. It samples keys, gives only random-oracle access to the adversary, and accepts iff verification succeeds.
`SUF_CMA.main`	The strong unforgeability analogue of EUF-CMA. It logs returned signatures and requires freshness of the full signed query. It is defined for completeness of the generic interface, although the HAETAE top-level theorem uses EUF-CMA/NMA.

The two freshness lemmas are proof-script sanity checks. `fresh_msg_nil` and `fresh_sig_nil` unfold the corresponding freshness predicate and solve membership in the empty list by simplification. No cryptographic assumption is used.

The lemma `nma_as_cma_exact` is the main proof block in this file. Its mathematical statement is

$$\Pr[\texttt{EUF\_CMA}(H, S, \texttt{NMA\_As\_CMA}(A))] = \Pr[\texttt{UF\_NMA}(H, S, A)].$$

The proof uses `byequiv` to compare two programs in lockstep. The pre- and postcondition `=\{globH,globS,globA\}==>=\{res\}` says that if the oracle, scheme, and adversary global states agree initially, then the two experiments return the same Boolean result. The `inline` step expands the adapter so the proof can see that it simply calls the NMA adversary. Each `call...bysim` aligns matching calls to the adversary, scheme, and random oracle. The final `auto=>/>` discharges the remaining first-order obligations, including that the unused signing oracle made no logged query.

This closes the script-companion coverage for `Sig_ROM.ec` at the level needed for the HAETAE proof: every declaration, game state, and proof block in the generic interface has a mathematical reading. Future additions would only be needed if later proofs begin using `SUF_CMA` in a security theorem.

1.15 Script companion coverage for HAETAE Reductions

`HAETAE_Reductions.ec` is the arithmetic ledger for the proof. It does not define a cryptographic game. Instead, it defines the real-valued loss and hardness terms that appear in the final theorem, then proves the algebraic and order facts needed to compose those terms.

Operation family	Mathematical reading
`mlwe_hardness_term`	Abstract upper bound for the success probability of the MLWE reduction module. It is not estimated in this file.
`module_sis_hardness_term`	Abstract upper bound for the Module-SIS reduction module. It is also not a concrete bit-security estimate.
`bimodal_to_msis_reduction_loss_term`	Loss slot for converting the bimodal self-target relation to Module-SIS. The current checked value is 0.
`bimodal_selftarget_msis_hardness_term`	The grouped bimodal hardness contribution: `module_sis_hardness_term` + `bimodal_to_msis_reduction_loss_term`.
`signature_query_count`, `hash_query_count`	The fixed counted-ROM wrapper constants $q_S = q_H = 2$. These are not symbolic adversarial query bounds.
`challenge_support_bits_lower_bound`, `challenge_support_cardinality_lower_bound`	The challenge-support lower bound 58 bits and 2^{58} elements, encoded as the concrete real 288230376151711744.
`counted_rom_collision_term`	Collision loss $(q_H + q_S + 1)^2/N = 25/2^{58}$.
`counted_rom_prequery_term`	Challenge prequery loss $q_S(q_H + 1)/N = 6/2^{58}$.
`counted_rom_min_entropy_term`	Counted min-entropy loss $q_S(q_H + 1)/N = 6/2^{58}$.
`counted_rom_programming_loss_term`	The counted-ROM sum $25/2^{58} + 6/2^{58} + 6/2^{58} = 37/2^{58}$.
`fs_with_aborts_*` and `rejection_sampling_loss_term`	The older paper-style Fiat-Shamir-with-aborts/rejection ledger. Its min-entropy component rewrites to 12, so it is tracked as a vacuity warning rather than a useful concrete loss.
`haetae_euf_bound`	Grouped legacy bound using the rejection-sampling ledger.
`haetae_euf_counted_rom_bound`	The final counted-ROM bound used by the top-level theorem.

The proof families in this file should be read as follows.

Lemma family	Proof-script reading
`haetae_euf_boundE`, `haetae_euf_bound_groupedE`, `haetae_euf_bound_rejection_groupedE`, `haetae_euf_counted_rom_bound_groupedE`	These lemmas unfold definitions and regroup real sums. The script pattern is `rewrite/definition...;ring`; mathematically, it proves only associativity, commutativity, and unfolding of named loss groups.

Nonnegativity lemmas	Lemmas such as `rom_hash_query_budget_nonnegative`, `counted_rom_collision_term_nonnegative`, and `rejection_sampling_loss_term_nonnegative` prove that probabilities and loss terms can be added or used in transitivity arguments without reversing inequalities. Tactics such as `addr_ge0`, `mulr_ge0`, and `divr_ge0` correspond to elementary real-order closure.
Support positivity lemmas	`challenge_support_cardinality_lower_bound_positive` and its nonnegative corollary justify divisions by 2^{58} and the use of `ltr_pdivr_mulr`.
Concrete counted-ROM rewrites	`counted_rom_collision_termE`, `counted_rom_prequery_termE`, `counted_rom_min_entropy_termE`, and `counted_rom_programming_loss_termE` compute the numerators 25, 6, 6, and 37. The scripts unfold query counts and use `ring`.
Subunit and support-dominance lemmas	`counted_rom_programming_loss_term_subunit`, `counted_rom_prequery_reprogramming_term_subunit`, and support dominance lemmas show the counted-ROM losses are less than 1. These facts are meaningful concrete sanity checks for the counted ledger.
Cardinality-summary lemmas	`counted_rom_loss_from_query_budgets_and_support` and related lemmas package both the formula for the loss and the strict domination of the numerator by the support size.
Vacuity lemmas	`fs_with_aborts_min_entropy_termE`, `fs_with_aborts_min_entropy_term_ge1`, and `rejection_sampling_loss_term_ge1` prove that the paper-style rejection-sampling ledger is already at least 1, with a min-entropy component exactly 12. This is why final claims must preserve the vacuity warning.
`bimodal_to_msis_reduction_loss_termE`	This confirms the current adapter-loss slot is exactly 0. It is a definition-unfolding fact, not a new hardness theorem.

This closes the script-companion coverage for `HAETAE_Reductions.ec` as an arithmetic ledger: every operation family and lemma family has a mathematical role and a proof-script reading. It does not close the cryptographic assumption coverage for MLWE or Module-SIS; those remain attached to `HAETAE_Assumptions.ec` and the final theorem composition.

1.16 Feedback 3 criticism for HopGames and Security

Before this expansion, the answer for `HAETAE_HopGames.ec` and `HAETAE_Security.ec` was still negative: a mathematician or cryptologist could not reliably understand those EasyCrypt scripts from the report alone, even with the scripts open. The evidence is internal to the document. The coverage table still marked the declaration dictionary, state interpretation, restriction explanations, lemma ledger, and proof-script commentary as partial. The existing `HAETAE_HopGames.ec` section only gave four implementation layers and a few representative lemma names; it did not explain the long chain of transcript games, counted-ROM disciplines, sampler bad events, erasure equivalences,

and exact-hyperball bridges. The existing `HAETAE_Security.ec` section correctly said that the file assembles the final theorem family, but it did not tell a reader how the successive composition lemmas differ, which premises are imported from hop games, which premises are hardness assumptions, and which steps are just checked real-order algebra.

The main logical risk was not that the EasyCrypt scripts were unchecked. The risk was that the prose made the proof look flatter than it is. In particular, the scripts distinguish:

- exact equivalences, such as erasing transcript logging or replacing one sampler wrapper by an extensionally equal wrapper;
- Hoare invariants, such as preservation of transcript soundness, bad-event flags, and query budgets;
- probabilistic inequalities, such as splitting a success event into a clean branch and a bad branch;
- external or conditional premises, such as MLWE hardness, Module-SIS hardness, and the structural-to-exact-hyperball sampling obligation;
- explanatory or coarse fallback facts, especially bounds that are true because `rejection_sampling_loss_term` is already at least 1.

Therefore, the revision below does not claim a new cryptographic theorem. It adds a companion map that explains what the machine checker proves in these two files, what remains a premise supplied by another checked file or an external assumption, and which statements are explanatory bridges or vacuity warnings.

1.17 Script companion coverage for HAETAE HopGames

`HAETAE_HopGames.ec` is the largest script because it turns the generic EUF-CMA game into transcript-aware and sampler-aware games. It is not a single game hop. It is a stack of games, invariants, exact equivalences, and bad-event bounds. The table below is the reading guide for its main proof layers.

Script layer	Mathematical and proof-script reading
`SigningSimulator`, `RealSigningSimulator`, `EUF_CMA_SimulatedSign`	Abstracts the signing oracle behind a simulator interface. The simulated game has the same EUF-CMA success predicate as `SIG.EUF_CMA`, but calls `Sim.sign` and records the queried message-context pairs. The checker verifies procedure calls and state updates; no hardness assumption is used.
`transcript_log_consistent`, `transcript_log_ready`, `internal_transcript_state_sound`	Defines the invariant connecting three views of signing history: `queries`, `transcripts`, and `records`. Read the lemmas ending in `_nil`, `_cons`, `_valid`, and `_no_min_entropy` as list invariant lemmas. They are checked structural facts about logs, not cryptographic reductions.

Budget predicates	`budgeted_programmed_query_count_discipline`, `budgeted_paper_sim_signing_count_discipline`, `budgeted_paper_sim_sampler_freshness_discipline`, and related predicates are the explicit query-budget conditions. They are the reason the counted-ROM terms in `HAETAE_Reductions.ec` can use the concrete constants $q_H = q_S = 2$.
`sampler_rom_covered`	States that every sampler-expand query already present in the lazy-ROM map is accounted for either as an adversary hash query or as a sampler query. The "fresh after clean seed" lemmas convert absence from the two logs plus a clean flag into absence from the ROM table.
Public-simulation operations	`public_sim_*` operations reconstruct the public pieces of a signing transcript from public coins and the public key. Their `_E` lemmas are definition-expansion facts used in equivalence proofs.
Paper-simulation sample operations	`paper_sim_*`, `paper_sim_sample_from_public_coins`, `paper_sim_sample_from_real_signing_coins`, and `paper_sim_sample_from_rejection_attempt` normalize several sampler representations to a common `paper_sim_signature_sample`. The checker proves field equalities and well-formedness; this is explanatory plumbing for later game hops.
Sampler modules	`ROMPaperSimSigningSampler`, `RealSigningPaperSimSampler`, `ROSigningAttemptPaperSimSampler`, `HAETAERejectionPaperSimSampler`, `ExactHyperballHAETAERejectionPaperSimSampler`, `ExactHyperballPaperSimSampler`, and `ROExactHyperballPaperSimSampler` are alternate implementations of the signing-sample source. The proof uses them as named endpoints in NMA games.
NMA wrappers	`ROMInternalTranscriptAsNMA`, `ROMInternalTranscriptPublicSimAsNMA`, `ROMInternalTranscriptPaperSimAsNMA`, and `ROMInternalTranscriptBudgetedPaperSimAsNMA` turn an adaptive adversary plus a simulated signing oracle into an NMA adversary that internally runs the transcript game. This is the central structural trick of the proof.
Counted and programmed transcript games	`EUF_CMA_ROMInternalTranscriptCountedSign`, `EUF_CMA_ROMProgrammedTranscriptCountedSign`, `EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign`, and `EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign` expose the random oracle bad flags, programmed sites, and bounded hash/sign counters.

Invariant sections	`InternalTranscriptSignSound`, `CountedROMInternalTranscriptDiscipline`, `ProgrammedROMTranscriptDiscipline`, `BudgetedProgrammedROMTranscriptDiscipline`, and `BudgetedSampledROMTranscriptDiscipline` prove that the game states stay sound after oracle calls and adversary calls. These are Hoare proofs: `proc`, `wp`, `call`, `rnd`, and `auto` discharge preservation of explicit predicates.
`BudgetedProgrammedToSampledDirectBridge`	Pdirect bridge between a counted game that samples signing coins directly and a sampled game using `DirectSigningCoinSampler`. The long relational postconditions state equality of all shared state fields and bad flags.
Erasure and exactness sections	`HAETAEROMInternalTranscriptExact`, `ROMInternalTranscriptStructuralNMA`, `ROMInternalTranscriptPublicSimNMA`, `ROMInternalTranscriptROMPaperSimNMA`, `ROMInternalTranscriptRealSigningPaperSimNMA`, `ROMInternalTranscriptROSigningAttemptPaperSimNMA`, `TranscriptLoggingErasure`, and `RealSigningSimulatorExact` are `byequiv` sections. They prove equality or one-sided implication of success events by matching calls and preserving corresponding state.
Exact-hyperball lifting layer	`structural_to_exact_hyperball_paper_sample_loss_obligation` is a conditional per-call sampling-loss premise. Lemmas around `concrete_ro_signing_attempt_to_exact_hyperball_*` transport this premise through lazy-ROM freshness and clean-event hypotheses.
Concrete `HAETAE_RO.FRO` layer	`ConcreteROMInternalTranscriptROSigningAttemptExactHyperballPaperSi` specializes the abstract oracle lifting to the concrete lazy random oracle. Several lemmas here still conclude with coarse probability bounds because the rejection term is already at least 1.
Rejection and no-fallback exactness	`ROMInternalTranscriptHAETAERejectionPaperSimNMA` and `ROMInternalTranscriptExactHyperballPaperSimNMA` connect rejection-sampling and exact-hyperball samplers. The "no fallback" lemmas rely on support facts about `dexact_hyperball_signing_attempt_state`.
Special-soundness interface layer	`BimodalSpecialSoundness_Game`, `SpecialSoundness_Game`, `BimodalTranscriptExtractor_As_ModuleSIS`, and the `special_soundness_interfaces_*` lemmas are interface checks for the forking/special-soundness boundary. They do not prove a new lattice theorem; they package obligations imported from the assumption and transcript files.

The main mutable state of `HAETAE_HopGames.ec` has the following mathematical interpretation.

State component	**Meaning**

`pk_current`, `sk_current`	The public and secret key currently installed in a transcript game or NMA wrapper. Invariants typically require `pk_current` = `public_key_of_secret(sk_current)`.
`queries`	The generic EUF-CMA signing log, containing only message-context pairs.
`transcripts`	The transcript log used by ROM programming and special-soundness reasoning. Validity and no-min-entropy-failure predicates are stated over this list.
`records`	The synchronized list of message, context, signature, and transcript records. It is the bridge between the generic signing log and the transcript log.
`C.hash_queries`	The counted lazy-ROM log of hash queries. Challenge-query counts are derived from this list.
`C.programmed_sites` and `C.signature_sites`	The random-oracle sites that have been programmed, and the sites induced by signing transcripts. Conflict-freedom of these sites is the ROM programming discipline.
`C.bad_prequery`, `C.bad_reprogram`, `C.bad_min_entropy`	Bad-event flags for prequerying the challenge site, attempting inconsistent reprogramming, and hitting a transcript min-entropy failure. The counted-ROM proof bounds these flags rather than hiding them in prose.
`adversary_hash_count`, `signing_count`	Budget counters. The scripts intentionally saturate them at the fixed budgets instead of leaving the number of queries symbolic.
`adversary_hash_queries`, `sampler_expand_queries`, `sampler_bad_prequery`	The budgeted paper-simulation NMA wrapper's sampler-specific query log and bad event. These variables support the split between clean sampling and sampler prequery failure.
Sampler `sk_current` fields	Sampler modules store the secret key needed to produce signing samples. The equivalence proofs require equality of these fields across paired samplers.

The proof-script patterns in this file should be read as follows.

Pattern	**Reader's interpretation**
`hoare[...]` with invariant preservation	The checker proves that each oracle or adversary call preserves an explicit state predicate. These blocks are not probability bounds; they are deterministic or almost-sure invariant arguments over program state.
`byphoare` converting Hoare facts to probabilities	Used for zero-probability bad-event lemmas such as transcript bad-forgery and min-entropy bad-event claims. The mathematical step is: if a Hoare proof shows the event is impossible, its probability is 0.
`byequiv` and long relational postconditions	The checker compares two games in lockstep. Equalities such as `=\{globH,globA\}` and field-by-field equalities are the formal statement that the two executions expose the same interface and return the same result, or that a left success implies a right success.

`Pr[mu_split]`, `Pr[mu_sub]`, `Pr[mu_or]`	These are probability algebra steps. They split a success event into clean and bad branches, drop conjuncts to obtain upper bounds, or union-bound bad events.
`ler_trans`, `ler_add`, and `smt(mu_bounded)`	These are real-order composition steps. They do not add cryptographic content; they combine already-proved inequalities and use that probabilities lie between 0 and 1.
Coarse fallback bounds	When a proof shows a desired inequality using `rejection_sampling_loss_term_ge1` and `mu_bounded`, the result is machine-checked but vacuous as a concrete-security estimate. The report must keep this distinction visible.

For `HAETAE_HopGames.ec`, the machine-checked content is the typing, Hoare judgments, equivalence judgments, probability inequalities, and real-order side conditions appearing in the script. The MLWE and Module-SIS hardness terms are not proved here. The structural-to-exact-hyperball paper-sample loss is a conditional premise when it appears as `structural_to_exact_hyperball_paper_sample_loss_obligation`. The prose explanations in this section are only a reading guide for those checked statements.

1.18 Script companion coverage for HAETAE Security

`HAETAE_Security.ec` is the composition file. It does not implement the sampler or ROM machinery itself. Instead, it imports the previous scripts and assembles theorem families by repeatedly applying already-proved hops, hardness premises, and arithmetic rewrites.

Section or lemma family	Mathematical and proof-script reading
`correctness_obligation` and `correctness_obligation_holds`	Defines perfect correctness as verification of every internally generated signature. The proof unfolds `keygen_internal` and applies `verify_internal_sign_internal`. This is checked internal correctness, not a security reduction.
`correctness_perfect`	Shows the generic correctness game fails with probability 0. The proof inlines the game and HAETAE procedures, uses Hoare reasoning, samples keys and coins, and discharges the final verification facts with `valid_signature_sign_internal` and `verify_norm_ok_current`.
`NoSigningSecurity`	Uses `SIG.nma_as_cma_exact` to embed an NMA adversary as a CMA adversary with no signing queries. The lemma composes an NMA-to-hardness premise, the bimodal-to-Module-SIS premise, and MLWE/Module-SIS hardness bounds into the legacy `haetae_euf_bound`.
Generic `euf_cma_security` lemmas	These are algebraic composition lemmas. Their inputs are a Fiat-Shamir hop bound, an NMA hardness bound, a bimodal-to-Module-SIS bound, and the two hardness-term bounds. The proof uses `ler_trans`, `ler_add`, and bound rewrites.

`FullReductionWithSimulatorHop`	Replaces the generic EUF-CMA game by `EUF_CMA_SimulatedSign` using `real_signing_simulator_exact`. The section is an exact-game interface bridge followed by the same real-order composition as the generic lemma.
`FullReductionWithROMTranscriptHop`	Main hop from ordinary signing to ROM-internal transcript signing. The `rom_internal_transcript_hop_from_clear_site_*` lemmas split success into a clear-site branch plus a bad-forgery branch and charge the bad branch to either the legacy FS-with-aborts terms or the counted-ROM programming term.
`MechanizedBimodalModuleSISLift`	`bimodal_to_module_sis_reduction_bound` rewrites the checked exact adapter `bimodal_msis_as_module_sis_exact` plus the zero adapter-loss term. It is not an independent proof of Module-SIS hardness.
`MechanizedROMTranscriptBimodalLift`	Combines the ROM transcript hop with the mechanized bimodal-to-Module-SIS adapter. The counted version targets `haetae_euf_counted_rom_bound`; the legacy version targets `haetae_euf_bound`.
`MechanizedROMTranscriptConstructedNMALift`	This is the NMA composition lane. It progressively replaces structural NMA, public-simulation NMA, paper-simulation NMA, real-signing paper simulation, HAETAE-rejection simulation, exact-hyperball simulation, and random-oracle exact-hyperball simulation, then applies MLWE and bimodal/Module-SIS bounds.
`euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal`	This is the representative checked counted-ROM theorem shape. Its premise is an NMA bound for `ROExactHyperballPaperSimSampler` plus `rejection_sampling_loss_term`. The conclusion is the formal EUF-CMA bound `haetae_euf_counted_rom_bound`.
`euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal`	This variant exposes the budgeted lifting premises explicitly: unbudgeted-to-budgeted for the signing-attempt sampler, budgeted-to-unbudgeted for the exact-hyperball sampler, the structural sampling-loss obligation, and a final NMA bound containing `rom_signature_query_budget` · `rejection_sampling_loss_term`.

The final composition section uses module restrictions to enforce information flow. The exclusions should be read by category, not as arbitrary syntax.

Restricted modules	**Why the restriction matters**
`SIG.EUF_CMA` and `EUF_CMA_*` games	Prevent the adversary or oracle from inspecting game-private secret keys, query logs, transcript logs, bad flags, or return events.
`ROMInternalTranscript*AsNMA` wrappers	Prevent circular access to the NMA wrappers that are being used as simulated adversaries in the reduction.

Paper-simulation sampler modules	Restrictions on `ROMPaperSimSigningSampler`, `RealSigningPaperSimSampler`, `ROSigningAttemptPaperSimSampler`, `HAETAERejectionPaperSimSampler`, `ExactHyperballHAETAERejectionPaperSimSampler`, `ExactHyperballPaperSimSampler`, and `ROExactHyperballPaperSimSampler` prevent adversaries from using sampler internals except through the declared signing-oracle interface.
`HAETAE`	When excluded from the adversary, prevents direct calls to the scheme module outside the intended oracle/game interface.
`H` and `A` exclusions in `Samp`	Ensure a sampler cannot call back into the random oracle or adversary except through the interfaces explicitly passed to it by the wrapper.
`Bmlwe`, `Bbimodal`, and `Bsis`	These are abstract reduction modules. Their success probabilities are bounded by premises or assumption terms; the security file composes those bounds but does not implement the reductions' internals.

The proof-script patterns in `HAETAE_Security.ec` are deliberately simple. A `rewrite` of an exactness lemma changes the game whose probability is being bounded without loss. A `rewrite` of a grouped bound such as `haetae_euf_counted_rom_bound_groupedE` unfolds the arithmetic ledger. An `apply` of a previous security lemma selects the next composition theorem. Nested `ler_trans` and `ler_add` blocks compose inequalities and preserve nonnegative loss terms. The checker verifies all of these real-order steps, but the cryptographic assumptions remain exactly the assumptions named in the premises: MLWE hardness, Module-SIS hardness, NMA-to-hardness bounds, and sampling-hop obligations.

Consequently, this file should be read as a machine-checked composition argument, not as a source of new assumptions. The statements that are machine-checked are the conditional theorem statements and their real-order proofs. The hardness estimates are assumed through the real-valued terms and module-game premises. The prose in this report explains that structure; it is not itself part of the EasyCrypt trusted base.

1.19 Feedback 3 criticism for assumptions, transcripts, and ROM programming

Before this expansion, the report still did not give enough evidence for a reader to understand `HAETAE_Assumptions.ec`, `HAETAE_Transcript.ec`, and `HAETAE_ROM_Programming.ec` at the level required by the new `HAETAE_HopGames.ec` and `HAETAE_Security.ec` companion sections. The old roadmap named the files and listed representative lemma names, but it did not explain which objects were external assumption interfaces, which were checked adapter equalities, which transcript predicates fed special soundness, and which random-oracle flags were later charged to counted-ROM loss terms.

That gap matters because the downstream proof scripts rely on these files without restating their mathematics. The hop-game script assumes the reader already knows the meaning of `valid_forking_pair`, `transcript_valid`, `actual_signature_programming_site`, `bad_prequery`, `bad_`

reprogram, and bad_min_entropy. The security script assumes the reader knows that MLWE_Game, ModuleSIS_Game, and BimodalSelfTargetMSIS_Game are assumption surfaces rather than proved cryptanalytic estimates. Without the companion material below, the report could still let a reader follow the names of the final theorem while missing the logical boundary between checked reductions, external hardness premises, conditional sampling premises, and explanatory prose.

1.20 Script companion coverage for HAETAE Assumptions

HAETAE_Assumptions.ec defines the formal hardness surfaces and the checked adapters between the bimodal self-target relation and Module-SIS. It does not estimate concrete MLWE or Module-SIS security. Its role is to give the rest of the proof typed games and exact interface adapters.

Script item	Mathematical and checker role
mlwe_instance, dmlwe_real, dmlwe_random	Abstract MLWE instance space and two lossless distributions. The script does not define their concrete algebraic distribution; it exposes a typed distinguishing game surface. Losslessness is checked from the declared [lossless] annotations.
module_sis_instance, module_sis_solution, module_sis_valid	Module-SIS instances are pairs (A, t), solutions are polyvecl vectors, and validity is the equation $A \cdot s = t$ plus well-formedness. The lemmas around module_sis_eval and module_sis_zero_valid are checked algebraic sanity facts.
bimodal_selftarget_msis_instance and solution aliases	The bimodal self-target relation is represented by the same carrier types as Module-SIS in this model. This is a formal interface decision, not a cryptanalytic theorem.
bimodal_to_module_sis_instance, bimodal_to_module_sis_solution, bimodal_to_module_sis_valid	Identity adapter from bimodal self-target MSIS to Module-SIS. EasyCrypt checks that validity is preserved by unfolding definitions.
MLWE_Distinguisher, MLWE_Real_Game, MLWE_Random_Game	Classical distinguishing surface for MLWE. These games sample from the abstract distributions and call an adversary. No hardness bound is proved here; bounds enter later as premises or as mlwe_hardness_term.
ModuleSIS_Solver and ModuleSIS_Relation_Game	Relation-game surface for a solver that outputs an optional Module-SIS solution. Success means sol<>None and module_sis_valid.
BimodalSelfTargetMSIS_Solver and relation game	Analogous relation-game surface for the bimodal self-target relation. The success event is later related exactly to Module-SIS through the identity adapter.
bimodal_solver_lift_exact and bimodal_mode_solver_lifted_distribution_exact	byequiv proofs showing that solving the bimodal relation and then viewing the result as Module-SIS has the same success probability as the lifted Module-SIS game. These are checked equivalences, not hardness estimates.

MLWE_Reduction, BimodalSelfTargetMSIS_Reduction, ModuleSIS_Reduction	One-procedure reduction interfaces used by the final theorem family. They are opaque modules whose success probabilities are bounded outside this file.
MLWE_Game, BimodalSelfTargetMSIS_Game, ModuleSIS_Game	Thin wrappers around the corresponding reduction modules. They exist so that security theorems can state probabilities uniformly.
BimodalMSIS_As_ModuleSIS and bimodal_msis_as_module_sis_exact	Final exact adapter used in HAETAE_Security.ec. The proof is a byequiv that calls the same underlying module on both sides.

The assumption boundary is therefore exact: checked material in this file includes type-correct games, adapter definitions, algebraic validity lemmas, losslessness of declared distributions, and equivalence lemmas for adapters. External material includes the actual hardness of MLWE, Module-SIS, and the bimodal self-target relation. The final security file may assume or compose bounds for Pr[MLWE_Game(...).main():res] and Pr[ModuleSIS_Game(...).main():res], but those bounds are not derived in HAETAE_Assumptions.ec.

1.21 Script companion coverage for HAETAE Transcript

HAETAE_Transcript.ec defines the transcript vocabulary used by random-oracle programming and special soundness. It is the bridge between a signature object and the public proof transcript extracted from that signature.

Transcript family	Mathematical reading
transcript	An eight-component tuple consisting of public key, message, context, commitment high bits, commitment low bits, message hash, challenge, and signature.
Projection operations	transcript_pk, transcript_message, transcript_context, transcript_commitment_highbits, transcript_commitment_lowbits, transcript_message_hash, transcript_challenge, and transcript_signature are tuple projections. Their role is readability and stable proof targets.
transcript_of_signature	Builds a transcript from a signature by extracting the commitment, message-hash, and challenge fields from the signature. This is used whenever signing games append a transcript to their log.
transcript_challenge_query and transcript_signature_challenge_query	Two views of the challenge random-oracle query: one from transcript fields and one from signature fields. Valid transcripts make them equal.
transcript_verifies, transcript_fields_match_signature, transcript_valid	Validity means the signature verifies, the signature is well-formed, and the stored transcript fields match the signature fields. This is the predicate used by ROM programming and special-soundness lemmas.

`transcript_challenge_matches` and `transcript_signature_challenge_matches`	Relate a random-oracle output to the transcript or signature challenge. Valid transcripts let the proof replace one predicate by the other.
`same_fork_target` and `distinct_fork_challenges`	Forking-pair structure: two transcripts must share public key, message, context, commitment high bits, commitment low bits, and message hash, while having different challenges.
`valid_forking_pair`	The exact precondition for extraction: same target, distinct challenges, and both transcripts valid. This avoids the logical error of claiming extraction from one accepting transcript.
`forking_difference_solution`, `fork_module_matrix`, `fork_module_target`	Build the bimodal self-target MSIS instance and candidate solution from a valid pair of transcripts. The solution is the response difference.
`extract_bimodal_selftarget_msis_solution` and `extract_module_sis_solution`	Partial extractors that return `None` unless the pair is valid. The Module-SIS extractor applies the adapter from `HAETAE_Assumptions.ec`.
Honest-transcript operations	`transcript_from_honest_signing`, `honest_transcript_verifies`, and `honest_transcript_challenge` connect deterministic internal signing to the transcript vocabulary consumed by ROM programming.

The lemma families in this file are proof-script glue for downstream scripts.

Lemma family	**What EasyCrypt checks**
Signature-field consistency	`transcript_fields_match_signature_challenge_query`, `transcript_fields_match_signature_challenge_matches`, `transcript_valid_signature_challenge_query`, `transcript_valid_signature_challenge_matches`, and `transcript_valid_signature_challengeE` unfold definitions to show that valid transcript fields and signature fields agree.
Public-equation and reconstruction lemmas	`transcript_valid_public_equation`, `transcript_valid_commitment_reconstruction`, `transcript_reconstructionE`, and `transcript_active_reconstructionE` justify using verification equations as public reconstruction equations in special-soundness reasoning.
Forking-pair query and challenge lemmas	`valid_forking_pair_challenge_query`, `valid_forking_pair_signature_challenge_query`, and `valid_forking_pair_distinct_signature_challenges` convert the abstract forking-pair predicate into equal challenge-query sites and distinct challenge values.

Extraction obligation lemmas	`extract_bimodal_forking_some`, `extract_bimodal_forking_nonempty`, `bimodal_forking_solution_valid`, and `forking_extraction_obligation_holds` prove that a valid forking pair produces a valid bimodal solution.
Module-SIS lift obligations	`bimodal_to_module_sis_lift_obligation_holds`, `module_sis_extraction_from_bimodal`, and `special_soundness_from_bimodal_lift` package the adapter from bimodal extraction to Module-SIS extraction.
Public reconstruction obligations	`fork_public_reconstruction_obligation_holds` and `special_soundness_with_public_reconstruction_from_bimodal_lift` record that public reconstruction accompanies extraction when later proofs need both facts.

The checked content here is definitional and algebraic: projections, tuple equality, validity propagation, extraction from a valid pair, and adapter validity. The explanatory boundary is important. This file does not prove a forking lemma that produces two transcripts from an adversary; it only states and checks what follows once a valid forking pair is available.

1.22 Script companion coverage for HAETAE ROM Programming

`HAETAE_ROM_Programming.ec` defines the counted random-oracle programming surface consumed by `HAETAE_HopGames.ec`. It is where transcript challenge sites become programmable ROM sites and where bad-event flags are named.

ROM-programming object	**Mathematical reading**
`programming_site`	A pair (q, y) consisting of a random-oracle query and the output to program at that query.
`programming_site_of_transcript`, `signature_programming_site_of_transcript`, `actual_signature_programming_site`	Ways to identify the challenge site induced by a transcript or by the signature fields inside that transcript. Validity lemmas show the transcript and signature views agree.
`honest_signing_programming_site`	The challenge site generated by honest signing coins. This is the random site whose freshness is needed for safe ROM programming.
`programmed_site_query_min_entropy_bound`	Point-probability bound for honest programming-site queries over signing coins. This is the min-entropy input used for prequery bounds.
`programming_site_matches_transcript` and `reprogramming_failure`	A programmed site is correct for a transcript when it has the transcript's challenge query and output matching the transcript challenge. Failure is the negation of that match.

challenge_entropy_support_ok, min_entropy_failure, transcript_log_min_entropy_clear	Challenge-support predicates. They are used to prove that honest transcripts do not trigger the min-entropy bad flag.
fs_with_aborts_failure and transcript_log_signature_programming_sites_clear	Legacy Fiat-Shamir-with-aborts failure combines reprogramming failure and min-entropy failure. The clear-log predicate says no transcript in the signing log fails at its actual signature programming site.
counted_rom_event and counted_trace_*	Event vocabulary for hash queries, signature sites, programmed sites, and min-entropy checks. These are counters and traces, not probability bounds by themselves.
ro_query_is_challenge and challenge_query_log_count	Classifies query constructors and counts distinct challenge queries in a log. Message, matrix-expansion, and sampler-expansion queries are not challenge queries.
programming_site_prequeried, programming_site_reprograms, programming_site_fresh	The three core bad-event predicates: a query was already asked, a site conflicts with an earlier programmed output for the same query, or neither bad event holds.
rom_counted_programming_bad and rom_counted_state_bad	Boolean summary of the counted bad state: min-entropy failure, prequery, or reprogramming conflict.

The stateful ROM module is central to the downstream proof.

CountedLazyROM state or method	Meaning
hash_queries	Log of queries passed to get. Challenge-query budgets are derived from this log.
programmed_sites	Sites passed to program. Reprogramming is detected by comparing a new site against this list.
signature_sites	Sites observed from signing transcripts. Hop games use this to connect transcript validity and ROM programming.
bad_prequery	Set when program(site) is called after the site's query already appears in hash_queries.
bad_reprogram	Set when program(site) conflicts with a previously programmed output for the same query.
bad_min_entropy	Set when observing a transcript whose challenge is outside the expected sparse support.
init, get, program, observe_signature_site, observe_transcript, bad	init clears logs and flags; get logs a query; program sets prequery/reprogram flags and calls the underlying oracle's set; observe_transcript records the induced signature site and updates the min-entropy flag; bad returns the disjunction of the three flags.

The lemma families needed by HAETAE_HopGames.ec and HAETAE_Security.ec are:

Lemma family	Proof-script reading
`CountedLazyROMStateFacts`	Hoare facts showing that `init` clears flags, `get` preserves clear flags or min-entropy clear state, `program` preserves bad-clear state when a site is fresh, and `bad` is false when all flags are clear. These are state invariants, not loss bounds.
Programming-site self-match lemmas	`programming_site_self_matches`, `programming_site_self_no_reprogramming_failure`, and `fs_with_aborts_failureE` unfold the definitions connecting a transcript, an oracle output, and a programming site.
Challenge-query counting lemmas	`challenge_query_log_count_cons_le` and the nonchallenge-cons lemmas show how query logs affect the counted challenge budget. These facts are used inside budgeted HopGames invariants.
Honest-site entropy and spread lemmas	`honest_signing_programming_site_query_entropy_token`, `signing_distribution_programming_site_query_spread`, and `honest_signing_programming_site_query_spread_bound` reduce programming-site min-entropy to the signing-coin token-spread assumption from the distribution layer.
Prequery probability bounds	`programmed_site_prequery_one_step_*`, `direct_signing_coin_sampler_prequery_bound`, and `direct_signing_coin_sampler_message_hash_prequery_bound` bound the probability that a freshly sampled signing site was already queried. These are the local ingredients for counted-ROM prequery terms.
Reprogramming conflict lemmas	`honest_signing_programming_sites_no_conflict`, `programming_site_log_conflict_free_for_honest_*`, `programmed_site_reprogram_one_step_conflict_free_zero`, and `programmed_site_prequery_reprogram_one_step_concrete_bound` show that honest sites do not conflict and combine reprogramming with prequery bounds.
Counted bad-flag budget lemmas	`counted_lazy_rom_bad_flags_budget_sound`, `counted_lazy_rom_bad_flags_query_budget_bound`, and `counted_lazy_rom_bad_flags_query_budget_bound_subunit` express the condition under which the probability of `CountedLazyROMBadGame` is charged to `counted_rom_programming_loss_term`.
Transcript no-failure lemmas	`honest_transcript_no_min_entropy_failure`, `signing_transcript_relation_no_min_entropy_failure`, `signing_record_log_sound_transcripts_no_min_entropy`, and `fs_with_aborts_no_failure_*` explain why honest signing records do not trigger the legacy failure predicate.

Valid-transcript programming-site lemmas	`transcript_valid_signature_programming_site_matches`, `transcript_valid_actual_signature_programming_site_matches`, and `valid_forking_pair_programming_site_query` connect transcript validity and forking pairs to identical programming-query sites.
Bound-obligation lemmas	`counted_rom_programming_bound_obligation_holds` and `fs_with_aborts_bound_obligation_holds` prove only nonnegativity of the named loss terms. They do not prove that the loss is small; the reductions file records separately that the old rejection ledger is vacuous.

The checked content in `HAETAE_ROM_Programming.ec` is therefore a set of definitions, Hoare invariants, and probability bounds for explicit bad events. The file does not justify replacing SHAKE by a random oracle, does not give a concrete MLWE/SIS estimate, and does not by itself complete the final EUF-CMA theorem. It supplies the exact bad-event vocabulary and counted-loss interfaces that HopGames and Security compose.

1.23 Feedback 3 criticism for the foundational dependency layer

The current report is still not enough for a mathematician or cryptologist to understand the EasyCrypt proof scripts from the report plus scripts alone at the layer below `HAETAE_HopGames.ec` and `HAETAE_Security.ec`. The missing material is not cosmetic. The hop and final-security files consume definitions from the formal HAETAE model as if they were ordinary mathematical objects: parameters, polynomial carriers, signature fields, random-oracle query constructors, signing-attempt distributions, rejection predicates, and concrete hash wrappers. Without a companion explanation for those files, a reader can see that a theorem compiles but cannot tell which part of the argument is an algebraic fact, which part is a model definition, which part is a conditional sampling premise, and which part is only explanatory idealization.

The main evidence from the scripts is as follows.

- `HAETAE_Security.ec` proves correctness through `verify_internal_sign_internal`, `valid_signature_sign_internal`, and `verify_norm_ok_current`. Those facts are not security assumptions; they are checked algebra/model facts from `HAETAE_Algebra.ec`.
- `HAETAE_HopGames.ec` repeatedly samples from `drandom_coins`, `dsigning_attempt_state`, and `dexact_hyperball_signing_attempt_state`. A reader must know that the current "exact hyperball" distribution is a checked bounded-product carrier, with comments in `HAETAE_Distributions.ec` marking the paper-faithful hyperball/sphere sampler as remaining correspondence work.
- The rejection bridge uses conditional premises such as `structural_to_exact_hyperball_paper_sample_loss_obligation` in `HAETAE_HopGames.ec` and obligation families in `HAETAE_Rejection.ec`. Those are not proved by the top-level theorem; they are explicitly consumed as premises or discharged only after accepting the current checked carrier/loss boundary.
- The scheme module in `HAETAE_Scheme.ec` makes exactly three random-oracle query kinds in signing: sampler expansion, message hash, and challenge hash. The ROM programming proof relies on this query discipline. Without the `HAETAE_ROM.ec` query/output dictionary, the bad-event and query-budget lemmas are opaque.

- `HAETAE_Algebra.ec` imports `HAETAE_FIPS202.ec`, which imports `HAETAE_Keccak1600.ec`. These files provide checked byte-list and size facts for the formal model, not a standard-model proof that SHAKE is a random oracle.

This section closes that gap at the level needed to read the scripts. It does not claim that the report alone can replace line-by-line EasyCrypt source. The checked source remains authoritative.

Category	**What belongs here in this layer**	**What must not be inferred**
Machine-checked facts	Definitions, typing, losslessness, support, size, well-formedness, equality, Hoare, equivalence, and probability inequalities proved inside the manifest files.	They do not automatically prove equality with the HAETAE paper specification, with C/Jasmin code, or with a concrete security estimate.
External cryptographic assumptions	MLWE, Module-SIS, and bimodal self-target MSIS hardness surfaces from `HAETAE_Assumptions.ec` and real-valued hardness/loss terms from `HAETAE_Reductions.ec`.	The foundational files below do not estimate these assumptions; they only define the formal carriers and adapters used by the reductions.
Conditional premises	Sampling and hop premises such as structural-to-exact hyperball lifting and random-oracle reprogramming bounds when they appear as antecedents of a security theorem.	They must not be presented as already discharged unless a checked lemma in the same manifest proves the exact antecedent.
Imported/library facts	EasyCrypt distribution, list, finite-map, real-order, and random-oracle library facts, plus the local Keccak/FIPS wrapper files.	Imported facts are part of the verification environment; a standalone proof archive must include the version and import contract.
Explanatory prose	Descriptions such as "formal HAETAE model", "bounded carrier", and "paper-shaped sampler interface".	Such prose is not a theorem. It guides reading and marks boundaries that the checked scripts either prove, assume, or leave conditional.

1.24 Script companion coverage for HAETAE Params

`HAETAE_Params.ec` is the numeric parameter base of the formal model. It has no cryptographic assumption and no probability theorem. Its machine-checked content is definitional arithmetic for modes and positivity facts consumed by algebra, distributions, rejection sampling, and final correctness.

Script object	Mathematical reading	Downstream role
`seedbytes`, `crhbytes`, `n`, `q`, `dq`	Global byte, polynomial-degree, and modulus constants. The checked fact `dqE` rewrites dq to 129026.	Used throughout the algebraic carriers, seed slicing, SHAKE wrappers, public key packing, challenge size, and distribution lengths.
`mode=[Mode2|Mode3|Mode5]`	The formal model has three parameter modes. There is no abstract well-formedness condition on modes beyond membership in this finite type.	`HAETAE_Scheme.ec` declares one abstract `haetae_mode`; all later security statements are mode-parametric through this value.
`mode_k`, `mode_l`, `mode_tau`	Mode-dependent vector dimensions and sparse-challenge weight prefix.	Used by matrix/vector dimensions, challenge sparsity, transcript extraction, and Module-SIS/bimodal relation shapes.
`mode_b0sq`, `mode_b1sq`, `mode_b2sq`	Mode-dependent squared-norm bounds for secret, response, and verification checks.	Used by `secret_norm_ok`, `response_norm_ok`, and `verify_norm_ok` in `HAETAE_Algebra.ec`, then by rejection-failure events and correctness.
`mode_ln`, `mode_alpha_hint`, `mode_m`	Formal signing-sample bound, hint-decomposition base, and $m = l - 1$.	Used by bounded/hyperball carriers, decomposition lemmas, public-key verification columns, and rejection-sampling thresholds.
`mode_crypto_bytes`, `mode_polyq_packedbytes`, `mode_publickeybytes`	Formal byte-length constants for signature and public-key encodings.	Used to prove packing well-formedness and that the pack branch in rejection sampling is never the abort source in the current model.
Positivity and size lemmas	`mode_k_gt0`, `mode_l_gt0`, `mode_tau_le_n`, `mode_crypto_bytes_gt0`, and related lemmas are checked by mode case analysis.	They discharge side conditions for `dlist`, `mkseq`, support predicates, norm arithmetic, and real-probability nonnegativity proofs.

Boundary: the parameter file checks internal arithmetic consistency of the formal constants. It does not prove that these constants are the claimed HAETAE standard parameter sets or that they yield a concrete bit-security level.

1.25 Script companion coverage for HAETAE Algebra

`HAETAE_Algebra.ec` is the formal HAETAE model core. It defines the carrier types and deterministic operations that the security games manipulate. The file is machine-checked, but its definitions are part of the model rather than a proof that the model is identical to the paper algorithm or an implementation.

Script object family	**Mathematical reading**	**Downstream role**
Carrier types	`byte`, `seed`, `crh`, `random_coins`, `message`, `context`, `coeff`, `poly`, `challenge`, `polyveck`, `polyvecl`, `matrix`, `pkey`, `skey`, and `signature` are list- and tuple-based carriers.	These are the concrete type instantiations used when `HAETAE_Scheme.ec` clones `Sig_ROM.ec`. All games and reductions share these carriers.
Zero values and well-formedness	`poly_zero`, vector zeroes, `matrix_zero`, `poly_wf`, `polyveck_wf`, `polyvecl_wf`, `crh_wf`, `challenge_wf`, and unit/sparse predicates define local shape constraints.	Used by signature validity, transcript validity, distribution support, and sampling support lemmas.
Polynomial and vector operations	`coeff_mod`, `poly_add`, `poly_mul`, `poly_dot`, `matrix_vec_mul`, `polyveck_add`, `polyveck_sub`, and `polyvecl_sub` define the algebraic operations over the formal list model.	The checked `_wf` and coefficient-bound lemmas make later Module-SIS and verification equations well typed and well shaped.
Decomposition and public-key packing families	`coeff_decompose_z1_*`, `poly_highbits`, `poly_lowbits`, `polyveck_vk_highbits`, `polyveck_highbits_hint`, packing/unpacking operations, and roundtrip lemmas describe the formal byte/list encoding layer.	Used by public-key reconstruction, verification-matrix construction, and transcript public-equation lemmas. These are checked formal encodings, not a complete C/Jasmin parsing theorem.
FIPS/SHAKE seed flow	`haetae_shake256_bytes`, `haetae_xof256_*`, `haetae_keygen_rhoprime`, `haetae_keygen_sigma`, and `haetae_keygen_key` wrap `HAETAE_FIPS202.ec`.	Used to define formal key-generation seeds and public matrices. This is checked byte-list plumbing, not a random-oracle theorem about SHAKE.

Key-generation relation	`public_matrix_seed`, `public_secret_vector_seed`, `public_error_vector_seed`, `public_key_vector_seed`, `public_key_relation`, `public_key_equation_holds`, and `public_key_of_secret` define the formal public key relation.	Used by `HAETAE_Transcript.ec` extraction, by Module-SIS instances, and by `HAETAE_Security.ec` correctness.
Signature fields	`valid_signature`, `sig_commitment_highbits`, `sig_commitment_lowbits`, `sig_message_hash`, `sig_challenge`, `sig_response`, `sig_response_aux`, and `sig_hint` define the tuple layout of a signature.	Used by transcript construction, challenge-query predicates, rejection attempt states, and verification.
Deterministic signing model	`commitment_raw`, `commitment_lowbits`, `message_hash`, `challenge_hash`, `commitment_challenge`, `commitment_highbits`, `response_vector`, `hint_vector`, and `signature_token` define signing fields from secret key, message, context, and coins.	Used by the real, public-simulation, paper-simulation, and rejection-simulation signing games in `HAETAE_HopGames.ec`.
Internal algorithms	`keygen_internal`, `sign_internal`, `verify_challenge_consistent`, `verify_public_equation`, `verify_norm_ok`, and `verify_internal` define the scheme core.	`HAETAE_Security.ec` correctness reduces directly to `verify_internal_sign_internal`. `HAETAE_Scheme.ec` exposes these operations through the generic signature interface.

The main checked lemma families are:

Lemma family	**Checked fact**	**Boundary**
Well-formedness and coefficient bounds	`poly_add_wf`, `poly_mul_wf`, `matrix_vec_mul_wf`, vector well-formedness lemmas, coefficient-bound lemmas such as `polyveck_add_coeffs_q_bound`, decomposition `_wf` lemmas, and poly-q bounds show that formal operations preserve shape and ranges.	Machine-checked.

Seed/XOF and packing facts	haetae_reference_keygen_xof_*, haetae_keygen_*_size, haetae_pack_vk_ref_roundtrip_*, haetae_public_key_roundtrip, and public-key byte well-formedness lemmas prove list-size and roundtrip facts for the model.	Machine-checked formal-model facts; implementation equivalence remains outside this file.
Public-equation facts	public_key_of_secret_relation, public_key_of_secret_equation_holds, public_key_vector_seed_mode23_keygen_equation, and public_key_vector_seed_mode5_keygen_equation prove the formal verification equation shape.	Machine-checked; used by transcripts and extraction.
Signature validity and correctness	valid_signature_sign_internal, verify_challenge_consistent_sign_internal, response_norm_ok_current, verify_norm_ok_current, and verify_internal_sign_internal.	Machine-checked formal correctness, used by HAETAE_Security.ec.

Boundary: this file contains no MLWE/SIS hardness theorem and no ROM theorem. It is the formal model that later security theorems prove statements about.

1.25.1 Line-by-line proof-script companion for HAETAE Algebra

Focused criticism for this pass: the preceding dependency-level table names the major object families, but it is not enough for a mathematician or cryptologist to reconstruct the proof obligations in HAETAE_Algebra.ec. In particular, it did not say where the script switches from model declarations to checked preservation lemmas, which preconditions are theorem hypotheses rather than global facts, which proof steps rely on EasyCrypt library lemmas such as size_mkseq, nth_mkseq, eq_from_nth, and integer SMT, or why the final verify_internal_sign_internal theorem is only a formal-model correctness theorem. The following line-range companion closes that gap at proof-obligation granularity. It is still not a replacement for reading the source: tactic sequencing and all generated subgoals remain in the .ec file and in EasyCrypt's checker.

Lines	Script content	Classification	Companion reading and downstream role

1–7	Imports and theory wrapper.	Imported/library facts.	`AllCore`, `IntDiv`, and `List` provide boolean logic, integer division/modulus, list constructors, `size`, `nth`, `nseq`, `mkseq`, `range`, `zip`, and `foldl`. `HAETAE_Params.ec` supplies all mode constants; `HAETAE_FIPS202.ec` supplies the formal SHAKE byte-list API. Any proof later using `smt`, `size_mkseq`, or SHAKE size lemmas imports those facts rather than reproving them here.
9–29	Carrier declarations for bytes, seeds, polynomials, vectors, matrices, keys, and signatures.	Machine-checked definitions.	These lines fix the formal universe: all algebra is over lists and tuples. They are checked type declarations, not a theorem that these carriers equal the HAETAE paper objects or Jasmin/C memory layouts. `HAETAE_Scheme.ec` and the generic `Sig_ROM.ec` clone use these exact carriers.
31–106	Zero objects, well-formedness predicates, challenge sparsity predicates, and their first size/well-formedness lemmas.	Machine-checked facts with imported list facts.	The proof pattern is definitional unfolding followed by `size_nseq`, `all_nseq`, `allP`, and `mem_nth`. These lemmas are the source of later side conditions saying that a formal polynomial has length n, a k-vector has `mode_kmd` rows, and a challenge has coefficients in $\{-1, 0, 1\}$.
108–169	Coefficient arithmetic, polynomial operations, vector operations, matrix-vector multiplication, and the list model of negacyclic multiplication.	Machine-checked definitions; explanatory model boundary.	The operations are definitions in the EasyCrypt model. The file checks shape and range facts about them later, but this block is not a mathematical proof that the list operations are isomorphic to a quotient-ring implementation. That correspondence would require a separate algebra or implementation equivalence theorem.
170–289	`_wf` preservation lemmas for arithmetic and matrix/vector operations.	Machine-checked facts relying on imported list facts.	The script repeatedly unfolds an operation, splits zero-special cases, proves the length of `mkseq` outputs, and uses induction for `foldl` in `poly_dot_wf`. These proofs discharge the routine obligations later needed by `valid_signature`, transcript predicates, and verification matrix construction.

291–425	`coeff_q_bound` and coefficient-range preservation lemmas.	Machine-checked facts; imported integer arithmetic.	The key obligation is that every arithmetic constructor returns coefficients modulo q. EasyCrypt's integer division/modulus library and SMT prove `coeff_mod_q_bound`; fold induction lifts that fact to dot products. These range facts feed public-key packing and mode-dependent coefficient packing bounds.
427–562	High/low-bit decomposition, hint high bits, verification-key high bits, and well-formedness of decomposition maps.	Machine-checked definitions and facts.	This block formalizes the bit-decomposition vocabulary used by public-key packing and verification reconstruction. The proofs check only list shape; numeric range refinements appear in the next packing block. Do not read this as a standalone theorem about the paper decomposition unless the definition is separately matched to the paper.
564–672	Seed-derived public-key polynomials, SHAKE/XOF wrappers, seed-buffer layout, and keygen seed slices.	Machine-checked definitions plus imported hash facts.	The script defines formal byte-list plumbing for the reference key-generation seed flow. Later lemmas use `HAETAE_FIPS202.ec` size/domain facts. This is not a ROM or SHAKE security assumption; it is deterministic byte-list modeling.
674–755	Formal keygen relation: matrix seed, secret/error vectors, rounding vector, mode-specific public-key vector, and `public_key_relation`.	Machine-checked definitions.	These are the definitions that extraction and verification use as the public equation. The theorem boundary is definitional: the relation is checked inside the model, while cryptographic hardness of finding alternate witnesses is handled outside this file by later reduction assumptions.
757–841	Reference keygen aliases and seed-flow predicate.	Machine-checked definitions and explanatory prose boundary.	The reference names make the script readable against HAETAE naming conventions. The file proves these aliases line up with the formal relation, but does not certify a C or Jasmin reference implementation.

841–902	Verification column/matrix construction, challenge embedding, public-key challenge term, and reconstructed highbits.	Machine-checked definitions.	These lines define what `verify_public_equation` later checks. They are used downstream by transcript public-equation predicates and rejection-attempt field consistency.
904–1265	Public-key byte encodings, simple pack/unpack definitions, reference 15-byte/16-bit packers, concrete packed-key aliases, and roundtrip predicates.	Machine-checked definitions; explanatory implementation boundary.	There are two encoding surfaces: a simple model packer and a reference-style bit packer. They are checked against their own definitions later. The presence of reference-style names should not be overread as a checked parser equivalence to every external implementation.
1267–1466	Seed/XOF and seed-buffer size/layout lemmas.	Machine-checked facts with imported hash/list facts and conditional premises.	Representative hypotheses include `0<=outlen`, `0<=len`, and `sizesd=seedbytes`. The proofs use unfolding, SHAKE size lemmas, and offset/slice arithmetic. The theorem boundary is deterministic size/layout correctness, not pseudorandomness.
1468–1767	Public matrix/vector well-formedness and poly-q range lemmas for formal and reference keygen.	Machine-checked facts with mode-case proof obligations.	The script proves that generated rows and vectors have the right lengths and coefficient bounds. Mode-specific cases `Mode2`, `Mode3`, and `Mode5` are split explicitly where packing ranges differ.
1769–1936	Verification-matrix well-formedness, seed pack/unpack, and byte-normalization lemmas.	Machine-checked facts relying on imported list/integer facts.	The important proof idioms are `eq_from_nth` for list equality, `nth_cat` for concatenated encodings, and integer SMT for byte normalization. These lemmas make the public-key roundtrip theorems possible.

1938–2753	Reference pack/unpack arithmetic for Mode2/3 15-byte blocks and Mode5 16-bit blocks; vector-body and public-key reference roundtrips.	Machine-checked facts with conditional premises.	The conditions `haetae_polyq_coeffs_wf`, `haetae_polyveck_polyq_coeffs_wf`, `md=Mode2\/md=Mode3`, and `sizesd=seedbytes` are local theorem hypotheses, not global truths. The proof decomposes byte offsets, proves each coefficient offset $0, \ldots, 7$, then lifts coefficient equality to polynomial and vector equality using `eq_from_nth`.
2771–3019	Simple model packing roundtrip, encoding well-formedness, and reference-key well-formedness.	Machine-checked facts with explicit hypotheses.	The simple packer proves a model roundtrip for `polyveck_wf` inputs; the reference packer proves a stronger poly-q-bounded roundtrip. These facts support keygen public-key recovery and verification-matrix construction in later games.
3022–3037	`public_key_of_secret_relation` and `public_key_of_secret_equation_holds`.	Machine-checked facts.	These are definitional correctness lemmas for honest keys. They feed transcript validity and extraction predicates, but they do not by themselves prove a hardness statement about producing another relation witness.
3039–3168	`valid_mode`, `valid_keypair`, `valid_signature`, signature projection functions, encoders, deterministic signing-field generators, and the signing entropy token extractor.	Machine-checked definitions; explanatory model boundary.	The formal signer is deterministic once coins are supplied. The low-bits field programs the first coefficient with the entropy token. This modeling choice is essential for ROM prequery and rejection-sampling arguments; it is not a claim about concrete entropy extraction unless separately connected to the implementation.
3171–3371	Well-formedness/unit/sparsity lemmas for deterministic signing fields and challenge hashes.	Machine-checked facts.	The proof pattern is constructor unfolding plus `mkseqP`, `allP`, and case analysis on parity and `i<mode_taumd`. These lemmas feed `valid_signature_sign_internal`, transcript validity, and rejection attempt acceptance.

3373–3404	`keygen_internal`, `sign_internal`, `verify_challenge_consistent`, `verify_reconstructed_highbits`, and `verify_public_equation`.	Machine-checked definitions.	This is the formal scheme core used by `HAETAE_Scheme.ec`. No probability, ROM, or adversary reasoning occurs here; those enter in `Sig_ROM.ec`, `HAETAE_HopGames.ec`, and `HAETAE_Security.ec`.
3407–3767	Keygen validity, public-key equations, reference keygen equations, packed-key roundtrips, verification-matrix correctness, and `valid_signature_sign_internal`.	Machine-checked facts with conditional premises.	Mode-specific equations split `Mode2`/`Mode3` from `Mode5`; roundtrip lemmas require `sizesd=seedbytes` or encoding well-formedness. The downstream role is to justify that honest formal keys and signatures satisfy the predicates consumed by transcript and security proofs.
3777–3883	Norm definitions, current-signing norm lemmas, and `verify_internal_sign_internal`.	Machine-checked facts; theorem boundary.	The final theorem proves: for this formal model, verifying an honestly signed message under `public_key_of_secretmdsk` succeeds. The norm proofs use unit-vector lemmas and mode case analysis. This is a correctness theorem for `sign_internal`/`verify_internal`, not HAETAE EUF-CMA security and not a statement about an external implementation.

The practical proof-script reading rule for this file is therefore: treat every `op` and `type` as a checked model declaration, every `lemma` as a checked theorem only under its displayed hypotheses, every use of `smt`/list rewriting as imported EasyCrypt reasoning, and every connection to the paper algorithm, SHAKE security, or implementation behavior as explanatory unless another source file proves it.

Algebra import and boundary ledger.

Category	Status inside `HAETAE_Algebra.ec`

Machine-checked facts	All displayed `lemma` statements in this file, including shape/range preservation, packing roundtrips, keygen equations, public verification-matrix facts, norm facts, and `verify_internal_sign_internal`.
Imported/library facts	`AllCore`, `IntDiv`, and `List` supply list equality, list indexing, sequence sizes, integer division/modulus, and SMT-dischargeable arithmetic. `HAETAE_Params.ec` supplies positivity and mode constants. `HAETAE_FIPS202.ec` supplies SHAKE size/rate/domain facts such as `shake256_size`, `shake256_squeeze_size`, `shake256_rate_bytesE`, and `shake256_domain_separatorE`.
External cryptographic assumptions	None are introduced locally. This file does not state MLWE, Module-SIS, ROM, Fiat-Shamir, or rejection-sampling security assumptions.
Conditional premises	No global cryptographic premise is assumed locally. Individual lemmas still have mathematical hypotheses, for example `sizesd=seedbytes`, `md=Mode2\/md=Mode3`, `polyveck_wfmdb`, or `haetae_polyveck_polyq_coeffs_wfmdb`; these hypotheses are theorem premises, not established facts about arbitrary inputs.
Explanatory prose boundary	Names containing "reference" or "concrete" identify formal EasyCrypt objects in this file. They do not by themselves certify a C/Jasmin implementation, the HAETAE standard document, or SHAKE pseudorandomness.

Final correctness dependency chain. The local correctness theorem decomposes exactly according to the conjunction in `verify_internal`. A reader replaying the proof should check these four obligations rather than treating `verify_internal_sign_internal` as an opaque correctness fact.

`verify_internal` conjunct	**Discharged by**	**Proof-script dependency reading**
`valid_signaturemdsig`	`valid_signature_sign_internal`.	Uses well-formedness of commitment highbits, lowbits, message hash, challenge, response, auxiliary response, and hint. These depend on deterministic constructor `_wf` lemmas and imported list-size facts.
`verify_challenge_consistentmdpkmctxsig`	`verify_challenge_consistent_sign_internal`.	Pure definitional unfolding of `sign_internal`, `commitment_challenge`, and `challenge_hash`; no hardness or probability reasoning enters.

verify_public_equationmdpksig	Direct rewrite inside verify_internal_sign_internal.	The proof expands verify_public_equation, verify_reconstructed_highbits, sign_internal, and commitment_highbits, then observes that the signature's auxiliary response field is the same value used to build the commitment highbits.
verify_norm_okmdsig	verify_norm_ok_current.	Uses the unit-vector lemmas for deterministic response, auxiliary response, and hint, then mode case analysis against the constants in HAETAE_Params.ec.

Packing and roundtrip obligation map.

Range	Main obligations	Premises and boundary
Mode2/Mode3 reference packer	The offset lemmas for ofs0 through ofs7 prove that each 15-byte block decodes eight 15-bit coefficients, then haetae_unpack_poly_q_ref_pack_mode23_coeff and vector-body lemmas lift coefficient equality to polynomials and vectors.	Requires md=Mode2\/md=Mode3 and haetae_polyq_coeffs_wf. Arithmetic is discharged by imported integer SMT plus byte-normalization lemmas.
Mode5 reference packer	haetae_unpack_poly_q_ref_pack_mode5_coeff, haetae_unpack_poly_q_ref_pack_mode5_at, and vector-body lemmas prove the 16-bit two-byte decoding roundtrip.	Requires haetae_polyq_coeffs_wfMode5. The checked bound is formal poly-q well-formedness, not an implementation parser theorem.
Simple model packer	haetae_unpack_poly_q_pack_at, haetae_unpack_polyveck_body_pack_at, and haetae_public_key_roundtrip.	Requires poly_wf or polyveck_wf. This packer is a model roundtrip surface distinct from the reference-style byte-packing surface.
Public-key recovery	haetae_keygen_public_key_roundtrip, haetae_keygen_public_key_ref_roundtrip, and verification-matrix correctness lemmas.	Requires seed-size or unpacked-key well-formedness premises. These lemmas justify downstream formal verification matrices, but not external implementation equivalence.

1.26 Script companion coverage for HAETAE Distributions

`HAETAE_Distributions.ec` defines the distributions used by key generation, random-oracle outputs, signing coins, and signing-attempt samples. Its most important boundary is the distinction between a checked bounded carrier and a paper-faithful exact HAETAE hyperball sampler.

Script object family	**Mathematical reading**	**Downstream role**
Byte, seed, CRH, coefficient, polynomial, matrix, and challenge distributions	`dbyte`, `dseed`, `dcrh`, `dcoeff`, `dpoly`, `dpolyveck`, `dpolyvecl`, `dmatrix`, and `dchallenge` are product or mapped distributions over the formal carriers.	`HAETAE_ROM.ec` uses these as random-oracle output components. `HAETAE_Scheme.ec` samples `dseed` and `drandom_coins`.
Signing randomness token family	`signing_entropy_token_cardinality`, `signing_entropy_token_to_coins`, `signing_randomness_domain`, `dsigning_entropy_token`, `drandom_coins`, and `signing_randomness_point_bound` encode a 58-bit entropy token embedded into signing coins.	Feeds the sampler-prequery and ROM min-entropy bounds in `HAETAE_HopGames.ec` and `HAETAE_ROM_Programming.ec`.
Signing sample pairs	`signing_sample_pair`, projections, `signing_sample_pair_wf`, `signing_sample_pair_unit`, `signing_sample_pair_bounded`, and `signing_sample_pair_sample_ok` define the shape of the two-vector sample used inside signing attempts.	`HAETAE_Rejection.ec` lifts these samples into full signing-attempt states and rejection predicates.
Structural and bounded sample distributions	`dstructural_signing_sample_pair`, `dbounded_unit_signing_sample_pair`, and `dchecked_hyperball_signing_sample_pair` provide deterministic, unit-product, and bounded-product sampling surfaces.	Used by hop games to relate real signing, paper simulation, and exact-hyperball sampling surfaces.
Hyperball interface	`hyperball_coeff_ok`, `hyperball_sample_ok`, `dhyperball_*`, point-bound operations, and `dexact_hyperball_signing_sample_pair` expose the sampler interface needed by rejection-sampling lifts.	The current checked implementation sets the exact hyperball interface to a bounded product distribution. The comments in the script mark the paper sphere/hyperball law as replacement/correspondence work.

Lemma family	Checked fact	Boundary
Losslessness lemmas	`byte_distribution_lossless`, `seed_distribution_lossless`, `signing_coin_distribution_lossless`, and hyperball/sample-pair losslessness lemmas show that each formal distribution has total mass 1.	Machine-checked.
Support/domain lemmas	`signing_coin_distribution_domain`, `signing_sample_pair_of_coins_sample_ok`, unit/bounded support lemmas, and hyperball support `P` lemmas connect sampling to shape predicates.	Machine-checked.
Point-bound lemmas	`signing_coin_distribution_token_point_bound`, `dlist_point_bound`, `hyperball_poly_point_boundE`, `hyperball_polyvecl_point_boundE`, and `exact_hyperball_signing_sample_pair_point_bound` provide the probability terms consumed by sampler-prequery bounds.	Machine-checked for the formal distributions.
Checked/exact equality	`exact_hyperball_signing_sample_pair_checkedE` proves that the current `dexact_hyperball_signing_sample_pair` equals the checked bounded carrier.	Machine-checked equality, but not a paper-sampler equivalence theorem.

Boundary: the file checks distribution properties for the formal carriers. It does not by itself prove that the formal signing randomness and bounded sample law are the final HAETAE paper distributions.

1.27 Script companion coverage for HAETAE Rejection

`HAETAE_Rejection.ec` is the bridge between deterministic signing fields, attempt-state sampling, transcript logs, and rejection-sampling loss terms. It is where many later hop premises get their vocabulary.

Script object family	Mathematical reading	Downstream role
Attempt-state records	`signing_transcript_record`, `signing_attempt_sample`, and `signing_attempt_state` package coins, sample vectors, commitment fields, challenge, response vectors, hint, and signature.	`HAETAE_HopGames.ec` samples and transforms these states in paper-simulation and exact-hyperball games.

Field reconstruction predicates	`signing_attempt_state_programmed_lowbits`, `signing_attempt_state_lowbits_consistent`, `signing_attempt_state_programmed_challenge`, `signing_attempt_state_public_equation_consistent`, and `signing_attempt_state_fields_match_signature`.	Used to prove that a sampled attempt can be read as a valid transcript and as a paper-simulation signature.
Rejection predicates	`signing_attempt_state_reject1_*`, `signing_attempt_state_reject2_*`, `signing_attempt_state_pack_*`, `signing_attempt_state_reference_rejection_*`, `signing_attempt_state_rejection_*`, and `signing_attempt_state_aborts` split the rejection event into named components.	Hop games charge or erase these events when moving from real signing to paper-simulation and exact-hyperball sampling games.
Attempt distributions	`dsigning_attempt_state`, `dstructural_signing_attempt_state`, `dbounded_unit_signing_attempt_state`, `dchecked_hyperball_signing_attempt_state`, and `dexact_hyperball_signing_attempt_state` lift sample-pair distributions to complete attempt states.	Security/hop lemmas compare success probabilities under these distributions.
Signing and transcript relations	`signing_attempt_relation`, `signing_simulation_relation`, `signing_transcript_relation`, `signing_record_relation`, `signing_record_log_sound`, and `transcript_log_valid` connect attempts to transcripts.	Used by transcript-log invariants in `HAETAE_HopGames.ec` and by ROM programming no-failure predicates.
Loss obligations	`structural_to_exact_hyperball_sample_pair_*`, `structural_to_exact_hyperball_coin_sample_pair_*`, `structural_to_exact_hyperball_attempt_loss_obligation`, and `rejection_sampling_bound_obligation` name the loss assumptions and nonnegativity premises.	Some are conditional premises for hop lemmas. The nonnegativity obligation is proved from `HAETAE_Reductions.ec` terms.

Lemma family	Checked fact	Boundary
Current-signing field equations	`signing_attempt_state_of_coins_signatureE`, `signing_attempt_state_of_coins_lowbitsE`, `signing_attempt_state_of_coins_challengeE`, `signing_attempt_state_of_coins_field_accepts`, `signing_attempt_state_of_coins_accepts_current`, and `signing_attempt_state_of_coins_rejection_accepts_current` prove that the formal current signer does not abort.	Machine-checked for the formal signer.
Sampler lifting and support	`dsigning_attempt_state_from_sample_pair_sampler_lossless`, `dexact_hyperball_signing_attempt_state_supportP`, and distribution support lemmas connect samples, coins, and full states.	Machine-checked for supplied sampler surfaces.
Abort decomposition and zero facts	`signing_attempt_state_reference_rejection_aborts_componentE`, union-bound lemmas, `dexact_hyperball_signing_attempt_state_reject1_abort_zero`, `dexact_hyperball_signing_attempt_state_reject2_abort_zero`, and related zero-abort lemmas show which rejection components vanish under the checked exact-hyperball attempt state.	Machine-checked; relies on the current exact-hyperball carrier definition.
Loss-obligation propagation	`structural_to_exact_hyperball_sample_pair_loss_from_point_loss`, `structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss`, and `structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss` lift a point/sample loss premise to an attempt-state loss premise.	Conditional unless the initial point-loss obligation is supplied.
Record-log soundness	`sign_internal_record_relation`, `signing_record_relation_sound`, `signing_record_log_sound_cons`, and `signing_record_log_sound_transcripts_valid` turn signing records into valid transcript logs.	Machine-checked; consumed by HopGames transcript invariants.

Boundary: this file is the precise place where rejection-sampling claims can become overclaimed. The checked scripts prove many zero-abort and propagation facts for the formal carrier. A paper-faithful sampler replacement still needs its own checked correspondence or must remain a conditional

premise.

1.27.1 Line-by-line proof-script companion for HAETAE Rejection

Focused criticism for this pass: the previous report named the rejection object families, but it did not let a reader reconstruct the proof boundary of `HAETAE_Rejection.ec`. It underexplained that many no-abort theorems are consequences of the current formal signer setting branch bytes to zero and of the current exact-hyperball support being lifted through `signing_attempt_state_of_sample_pair`; it also did not expose the conditional loss-obligation chain from point loss to full attempt-state loss. The line-range companion below records the proof obligations that a reader must track when replaying the EasyCrypt script.

Lines	**Script content**	**Classification**	**Companion reading and downstream role**
1–14	Imports, theory wrapper, and real-order import.	Imported/library facts.	`Distr`, `Real`, `StdOrder`, and `RealSeries` provide probability mass `mu`, support, `dmap`, `dlet`, union bounds, summability, and nonnegative real arithmetic. `HAETAE_Algebra.ec`, `HAETAE_Distributions.ec`, `HAETAE_Events.ec`, `HAETAE_Reductions.ec`, and `HAETAE_Transcript.ec` provide the domain-specific predicates used below.
16–96	Attempt records, samples, state tuple layout, projections, rejection-branch byte/bit, and reject2 balance vectors.	Machine-checked definitions.	These declarations define the state shape that hop games manipulate. The branch bit is read from signature component 6; because `sign_internal` sets that component to zero, later current-signing and sample-pair no-abort lemmas can prove reject2 is clear.
102–163	Programmed lowbits, lowbits consistency, programmed challenge, reconstructed highbits, public-equation consistency, signature reconstruction, and field acceptance.	Machine-checked definitions.	This block states the field-equality obligations needed to recognize a sampled attempt as a valid formal signature/transcript. Downstream `HAETAE_HopGames.ec` invariants use these predicates when comparing real signing, simulated signing, and ROM-programmed transcripts.

166–199	`signing_attempt_state_of_coins`, `signing_attempt_sample_of_pair`, and `signing_attempt_state_of_sample_pair`.	Machine-checked definitions.	The first constructor builds the current formal signing attempt from coins. The second and third constructors replace the sampled pair while keeping the deterministic signing fields from `HAETAE_Algebra.ec`. This is the bridge used by structural, bounded, checked-hyperball, and exact-hyperball attempt distributions.
203–339	Acceptance predicates, reject1/reject2/pack/abort rejection predicates, verification acceptance, full rejection acceptance, abort predicates, and `signing_accepts`.	Machine-checked definitions; exploratory rejection boundary.	The script decomposes rejection into named events. These definitions are not probability bounds by themselves; bounds appear only in later lemmas using `mu`. The names are the event vocabulary consumed by hop-game failure terms.
345–421	Attempt-state distributions and sample-pair shape predicates.	Machine-checked definitions plus imported distribution interface.	`dsigning_attempt_state` maps `drandom_coins`; the other distributions use `dlet` over coins and then map a sample-pair distribution into a full attempt. Losslessness and support are proved later, conditional on sampler-ok predicates supplied by `HAETAE_Distributions.ec`.
438–488	Signing relation, simulation relation, transcript relation, record projections, record log soundness, and transcript-log validity.	Machine-checked definitions.	These predicates connect rejection attempts to transcript reasoning. The security games use them to say that a logged signing query corresponds to a valid formal transcript under the public key.

491–523	Point, sample-pair, coin-sample-pair, and attempt-state loss obligations, plus nonnegativity obligation.	Conditional premises and machine-checked definitions.	`structural_to_exact_hyperball_sample_pair_point_loss_obligation` is not proved in this file. It is a conditional premise used to derive the larger distribution-comparison obligations. By contrast, `rejection_sampling_bound_obligation` is later proved from nonnegativity lemmas imported from `HAETAE_Reductions.ec`.
525–569	`signing_attempt_verifies`, `signing_simulation_verifies`, `signing_attempt_simulates`, and `sign_internal_attempt_relation`.	Machine-checked facts.	These lemmas lift `HAETAE_Algebra.ec`'s `verify_internal_sign_internal` into the rejection relation vocabulary. The proof obligation is mostly unfolding `valid_keypair` and applying formal correctness and norm lemmas.
571–623	Current sample-pair equality and sampler-ok lemmas for structural, bounded, checked-hyperball, and exact-hyperball samplers.	Machine-checked facts relying on imported distribution facts.	The sampler-ok lemmas are adapters: they expose the distribution-ok facts from `HAETAE_Distributions.ec` in the shape required by `dsigning_attempt_state_from_sample_pair_sampler`.
625–747	Losslessness and support/sample-ok lemmas for lifted attempt distributions.	Machine-checked facts with imported distribution rules.	The proofs use `dlet_ll`, `dmap_ll`, `supp_dlet`, and `supp_dmap`. They show that if the underlying sampler is ok, then the full attempt distribution is lossless and supported on well-formed bounded sample pairs.

749–799	Exact-hyperball support equivalence, one-bound, and pullback identity.	Machine-checked facts; model boundary.	`dexact_hyperball_signing_attempt_state_supportP` expands support into an existential over coins and a hyperball sample. This is the key lemma used repeatedly to turn a probabilistic zero-abort goal into a pointwise no-abort proof. It inherits the current formal definition of `dexact_hyperball_signing_sample_pair`; it does not prove paper-sampler faithfulness.
801–850	Reference-rejection abort decomposition and union bound.	Machine-checked facts with imported probability facts.	`signing_attempt_state_reference_rejection_aborts_componentE` rewrites the negation of a conjunction into a disjunction. The union-bound lemma uses `mu_or` and `mu_bounded`; these are library probability facts, not HAETAE-specific theorems.
852–964	Pack no-abort for sample-pair states and equality of structural/exact coin-sample-pair mappings.	Machine-checked facts.	Pack acceptance follows from `valid_signature_sign_internal` and `mode_crypto_bytes_gt0`. Distribution equalities are functorial rewrites using `dmap_dunit`, `dlet_dunit`, `dmap_dlet`, and `dmap_comp`.
966–1053	`mu_dlet_le_add_all` and the loss-obligation propagation chain.	Machine-checked facts conditional on the earlier point/sample loss premise.	The probabilistic lemma proves that a per-coin distribution inequality with additive ϵ lifts through `dlet`. The three propagation lemmas then derive sample-pair loss, coin-sample-pair loss, and full attempt-state loss. The initial point-loss premise remains conditional unless separately proved.
1055–1270	Field-equality lemmas for `signing_attempt_state_of_coins` and current field acceptance/no-abort.	Machine-checked facts.	The proofs are definitional rewrites showing that every projection of the attempt state equals the corresponding deterministic field from `HAETAE_Algebra.ec`. These lemmas are the backbone of current-signing transcript validity and current no-abort facts.

1272–1513	Reject1 bound lift, reject2 concrete rewrites, branch-clear proofs, sample-pair reject1/reject2/pack/hint no-abort, and exact-hyperball reference-rejection abort-zero chain.	Machine-checked facts with imported probability facts.	The pointwise no-abort lemmas rely on `sign_internal`'s branch byte being zero and on formal current norm bounds. The distribution zero lemmas use `dexact_hyperball_signing_attempt_state_supportP`, `mu_le`, `mu0`, and `mu_bounded`. The conclusion is zero abort in the current formal exact-hyperball attempt model, not a paper-level rejection-rate theorem.
1515–1672	Current-coin reject2 branch clearing, balance identities, reference-rejection acceptance, verification acceptance, full rejection acceptance, and full current no-abort.	Machine-checked facts.	These lemmas show that attempts generated directly from current signing coins pass all formal acceptance predicates. They are used later to prove zero abort probability for `dsigning_attempt_state`.
1674–1724	`dsigning_attempt_state` losslessness, entropy-token spread, and current reference/full rejection abort-zero probabilities.	Machine-checked facts with imported distribution facts and a distribution point-bound imported from `HAETAE_Distributions.ec`.	The token-spread lemma maps the signing-coin token bound through the attempt state. This feeds sampler-prequery and ROM bad-event arguments in `HAETAE_HopGames.ec`.
1726–1838	Simulation relation, transcript validity, transcript relation, record relation, record-log soundness, and query/transcript list constructors.	Machine-checked facts.	These lemmas connect accepted signing attempts to transcript logs. They are used by hop games and ROM programming to maintain that logged signing transcripts are valid and field-consistent.
1840–1854	Nonnegativity facts for rejection and Fiat-Shamir-with-aborts loss terms.	Machine-checked facts relying on imported reduction-term lemmas.	`rejection_sampling_bound_obligation_holds` is checked here, but the meaning of the loss terms comes from `HAETAE_Reductions.ec`. It proves nonnegativity only, not smallness or cryptographic tightness.

1856–2130	Field-equality and no-abort lemmas for `signing_attempt_state_of_sample_pair`.	Machine-checked facts.	This is the sample-pair analogue of the current-coins field-equality block. Because the deterministic signature fields still come from `sign_internal`, the proofs again reduce to definitional rewrites and the formal correctness/norm lemmas from `HAETAE_Algebra.ec`.
2132–2202	Exact-hyperball support implies reference rejection acceptance, full rejection acceptance, no-abort predicates, and zero full-rejection abort probability.	Machine-checked facts with model boundary.	The proof repeatedly expands exact-hyperball support into coins/sample witnesses and rewrites the state to the sample-pair constructor. The final zero-probability theorem is therefore conditional on the current formal exact-hyperball support definition; replacing that sampler requires replaying or reestablishing these support-to-no-abort arguments.

For downstream use, the theorem boundary is as follows. Machine-checked: state constructors, field predicates, support/lossless facts, current-formal no-abort facts, record-log soundness, and the conditional propagation from sample-pair loss to attempt-state loss. Conditional: the initial structural-to-exact hyperball point-loss obligation unless separately proved. Imported: probability algebra, list algebra, real summability, and the nonnegativity of loss terms from `HAETAE_Reductions.ec`. External or explanatory: the claim that the exact-hyperball carrier is the paper HAETAE sampler, the cryptographic interpretation of the loss terms, and any implementation-level rejection-sampling equivalence.

Rejection obligation sheet. The four loss obligations in lines 491–519 are not interchangeable. The script proves implications between them, but the first one remains the entry-point premise unless another file supplies it. To keep the formulas readable, write D_s^{str} for `dstructural_signing_sample_pair`, D_s^{ex} for `dexact_hyperball_signing_sample_pair`, D_c^{str} and D_c^{ex} for the structural and exact coin-sample-pair distributions, D_a^{str} for `dsigning_attempt_state`, D_a^{ex} for `dexact_hyperball_signing_attempt_state`, S for `signing_sample_pair_of_coins`, and L_{rej} for `rejection_sampling_loss_term`. With that shorthand, the EasyCrypt obligations are:

$$
\begin{aligned}
O_{\text{point}} &\quad \forall md, sk, m, ctx, coins.\ 1 \le \mu(D_s^{ex}(md))(\text{pred1}(S(md, sk, m, ctx, coins))) + L_{rej},\\
O_{\text{sample}} &\quad \forall md, sk, m, ctx, coins, p.\ \mu(D_s^{str}(md, sk, m, ctx, coins))(p) \le \mu(D_s^{ex}(md))(p) + L_{rej},\\
O_{\text{coin}} &\quad \forall md, sk, m, ctx, p.\ \mu(D_c^{str}(md, sk, m, ctx))(p) \le \mu(D_c^{ex}(md, sk, m, ctx))(p) + L_{rej},\\
O_{\text{attempt}} &\quad \forall md, sk, m, ctx, p.\ \mu(D_a^{str}(md, sk, m, ctx))(p) \le \mu(D_a^{ex}(md, sk, m, ctx))(p) + L_{rej}.
\end{aligned}
$$

Obligation	**Status in `HAETAE_Rejection.ec`**	**Downstream reading**

O_{point}	Conditional premise.	This is the carrier-level point-mass statement from which the script can derive larger loss comparisons. It must not be reported as proved by `HAETAE_Rejection.ec`.
O_{sample}	Machine-checked implication from O_{point}.	`structural_to_exact_hyperball_sample_pair_loss_from_point_loss` uses the structural sample's `dunit` form and monotonicity of `mu`.
O_{coin}	Machine-checked implication from O_{sample}.	`structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss` uses `mu_dlet_le_add_all` to lift the bound through coins.
O_{attempt}	Machine-checked implication from O_{coin}.	`structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss` uses distribution equalities that map coin/sample pairs into full attempt states.
`rejection_sampling_bound_obligation`	Machine-checked locally from imported nonnegativity lemmas.	This proves only nonnegativity of the three named loss terms. It does not prove that the losses are small or that the exact-hyperball law is paper faithful.

Carrier-specific theorem boundary.

Carrier or distribution	Checked theorem surface	Boundary
`signing_attempt_state_of_coins`	All current-field equalities, current acceptance, current rejection no-abort, zero abort probability for `dsigning_attempt_state`, and token-spread bound.	Machine-checked for the formal current signer and formal `drandom_coins`. Does not by itself prove paper rejection-sampling statistics.
`signing_attempt_state_of_sample_pair`	Field equalities, reference rejection acceptance, verification acceptance, full rejection acceptance, and no-abort predicates for any supplied sample pair.	Machine-checked because the deterministic signature fields still come from `sign_internal`. If a future sampler changes those fields, this boundary must be revisited.

`dexact_hyperball_signing_attempt_state`	Support characterization, pack/reject1/reject2/hint zero-abort probabilities, reference rejection zero abort, and full rejection zero abort.	Machine-checked for the current formal exact-hyperball carrier. The paper hyperball/sphere sampler correspondence is outside this file unless separately proved.
Paper HAETAE sampler	No direct theorem in this file.	External or conditional. The report must not claim that these zero-abort lemmas certify the paper sampler without a checked replacement/correspondence argument.

Two nontrivial proof blocks. For `mu_dlet_le_add_all`, the mathematical content is an additive-loss lifting rule: if every branch distribution $F_1(x)$ is bounded by $F_2(x)+\epsilon$, and $\epsilon \geq 0$, then the `dlet`-mixtures obey the same additive bound. The proof expands `dletE`, proves summability of the weighted series using imported `summable_mu1_wght`, `summableD`, and `summableZ`, applies `RealSeries.ler_sum`, and uses `weightE` plus `mu_bounded` for the final real-arithmetic side condition.

For the abort-zero chain, the pattern is pointwise-to-probabilistic. The script first proves that sample-pair states cannot trigger the relevant abort predicate. Then, under exact-hyperball support, every sampled state is rewritten to such a sample-pair state. Finally `mu_le` bounds the abort event by `pred0`, `mu0` rewrites the probability of `pred0` to zero, and `mu_bounded`/SMT closes the equality. This pattern proves zero probability for the current formal carrier; it is not a paper-level rejection rate proof.

Lemma-ledger criticism for HAETAE Rejection. Even after the line-range companion above, the report plus `HAETAE_Rejection.ec` still required too much reverse engineering for proof-obligation reconstruction. The missing layer was a ledger that says, for each security-relevant definition or lemma surface, what the checked statement is, what premises or imported facts it depends on, and how it feeds `HAETAE_HopGames.ec` or `HAETAE_Security.ec`. The ledger below adds that layer. Pure tuple projections and repeated field-equality siblings are grouped when they share the same mathematical proof obligation; the row lists the relevant EasyCrypt names and line ranges so the source remains the authority for exact syntax and tactic order. In particular, the ledger names the token/prequery bridge, the abort-channel predicates, the current-formal zero-abort theorems, the transcript/log bridge theorems, and the exact-hyperball support-to-no-abort interface, because these are the surfaces later games can actually cite.

Definition ledger for HAETAE Rejection.

EasyCrypt object(s)	Lines / category	Statement paraphrase	Premises, imports, downstream role
`signing_transcript_record`, `signing_attempt_sample`, `signing_attempt_state`	16–20. Machine-checked definitions.	Tuple carriers for logged signing transcripts, sampled vectors, and full signing-attempt state.	No premises. Imported carriers from `HAETAE_Algebra.ec`. HopGames uses these as the state space for rejection and transcript-signing games.

`signing_attempt_state_*` projections, `signing_attempt_sample_*`, `signature_rejection_branch_byte`	22–70. Machine-checked definitions.	Projection vocabulary for coins, sample components, commitment fields, challenge, response, hint, signature, and reject2 branch bit.	No premises. Tuple projection only. The reject2 branch reads signature field 6; this matters because `sign_internal` writes zero there.
`polyvecl_double`, `signing_attempt_state_reject2_sampled_*`, `signing_attempt_state_reject2_balance_*`	73–91. Machine-checked definitions.	Defines the formal reject2 balance vectors $2z - y$ and $2z_{\mathrm{aux}} - y_2$.	Imports formal vector arithmetic from `HAETAE_Algebra.ec`. HopGames uses these predicates when charging or erasing reject2 aborts.
`signing_attempt_state_lowbits_carrier`, `signing_attempt_state_token`, `signing_attempt_state_programmed_lowbits`, `signing_attempt_state_lowbits_consistent`	93–114. Machine-checked definitions.	Names the lowbits carrier and signing-token triple, then reconstructs lowbits by placing the signing entropy token in coefficient zero and requiring equality with the attempt state's lowbits.	Depends on `signing_entropy_token_of_coins`. Used by transcript and ROM programming invariants and the token point-bound lemma that bind sampler coins to challenge input.
`signing_attempt_state_programmed_challenge`, `signing_attempt_state_challenge_consistent`	116–128. Machine-checked definitions.	Recomputes the challenge from response auxiliary data, lowbits, and message hash under the public key.	Imports `challenge_hash` and `message_hash`. Used to show sampled attempts match the same challenge predicate as verification.
`signing_attempt_state_reconstructed_highbits`, `signing_attempt_state_public_equation_consistent`	130–139. Machine-checked definitions.	Rebuilds highbits from auxiliary response and public-key challenge term, then requires equality with the attempt commitment.	Imports public-key challenge operations from `HAETAE_Algebra.ec`. Feeds transcript validity and simulated-signing soundness.
`signing_attempt_state_signature_of_fields`, `signing_attempt_state_fields_match_signature`, `signing_attempt_state_field_accepts`	141–163. Machine-checked definitions.	Assembles the signature tuple from attempt fields and combines lowbits, challenge, public-equation, and signature-field consistency.	Imports signature layout from `HAETAE_Algebra.ec`. HopGames uses this as the field-consistency side condition for accepted sampled transcripts.
`signing_attempt_state_of_coins`, `signing_attempt_sample_of_pair`, `signing_attempt_state_of_sample_pair`	166–199. Machine-checked definitions.	Constructs current signing attempts from coins, or attempts that replace only the sampled pair while keeping deterministic signer fields.	Imports `sign_internal`, `commitment_*`, `response_*`, and `hint_vector`. These constructors are the bridge between real signing, structural attempts, and exact-hyperball attempts.
`signing_attempt_state_valid_signature_accepts`, `signing_attempt_state_response_norm_accepts`, `signing_attempt_state_hint_norm_accepts`, `signing_attempt_state_accepts`	203–218. Machine-checked definitions.	Defines the non-rejection validity and norm acceptance components.	Imports `valid_signature`, `response_norm_ok`, and `verify_norm_ok`. Used by current no-abort and transcript soundness lemmas.

signing_attempt_reject1_bound, signing_attempt_reject2_bound, signing_attempt_reject2_balance_norm_sq, signing_attempt_reject2_concrete_aborts	221–238. Machine-checked definitions.	Defines reject1/reject2 thresholds and the concrete reject2 event $branch \wedge \|2z - y\|^2 < B_0\ell_n^2$.	Imports mode bounds from HAETAE_Params.ec and norm functions from HAETAE_Algebra.ec. Used in the rejection-abort decomposition in HopGames.
signing_attempt_state_reject1_*, signing_attempt_state_reject2_*, signing_attempt_state_pack_*, signing_attempt_state_reference_rejection_*, signing_attempt_state_rejection_*, signing_attempt_state_response_norm_aborts, signing_attempt_state_hint_norm_aborts, signing_attempt_state_rejection_sampling_aborts, signing_attempt_state_aborts, signing_accepts	240–343. Machine-checked definitions.	Names the reject1, reject2, pack, reference-rejection, verification, full rejection, concrete abort-channel, coarse abort, and signing-acceptance predicates.	No cryptographic assumption locally. These event predicates are the exact bad-event and loss vocabulary used in HopGames; the report must not treat them as probability bounds until later mu lemmas are applied.
dsigning_attempt_state, dsigning_attempt_state_from_sample_pair_sampler, dstructural_*, dbounded_*, dchecked_*, dexact_*	345–419. Machine-checked definitions.	Defines distributions over attempts by mapping signing coins and sample-pair distributions into full states.	Imports drandom_coins and sample-pair distributions from HAETAE_Distributions.ec. Used by hop probability comparisons and sampler-prequery arguments.
signing_attempt_state_sample_pair_wf, signing_attempt_state_sample_pair_unit, signing_attempt_state_sample_pair_bounded	421–435. Machine-checked definitions.	Support predicates for sampled y_1, y_2 components.	Imports polyvecl_wf, polyveck_wf, unit and bounded predicates. Used by support lemmas and sampler-correctness checks.
signing_attempt_relation, signing_simulation_relation, signing_transcript_relation	438–456. Machine-checked definitions.	Relates a signature to an honest attempt, wraps it as a verified simulation, and then as a valid transcript.	Imports valid_keypair, verify_internal, and transcript predicates. Used by NMA transcript construction and signing-log soundness.
signing_record_*, signing_record_log_sound, transcript_log_valid	459–488. Machine-checked definitions.	Projects record fields, lists record queries/transcripts, and states log soundness/validity.	Imports list map and transcript validity. HopGames consumes these to preserve transcript logs across signing-oracle games.

structural_to_exact_hyperball_*obligation, rejection_sampling_bound_obligation	491–523. Conditional premises plus checked definitions.	States the point, sample-pair, coin-sample-pair, attempt-state loss obligations and the nonnegativity obligation.	O_{point} is not proved here. Nonnegativity is proved later from HAETAE_Reductions.ec. These obligations feed the exact-hyperball transition in HopGames and the final security bound.

Lemma ledger for HAETAE Rejection.

EasyCrypt lemma(s)	Lines / category	Statement paraphrase	Premises, imports, downstream role
signing_attempt_verifies, signing_simulation_verifies, signing_attempt_simulates, sign_internal_attempt_relation	525–569. Machine-checked facts.	Honest signing attempts verify, simulation relations verify, attempts lift to simulation relations, and sign_internal forms an honest attempt.	Imports verify_internal_sign_internal, valid_signature_sign_internal, response_norm_ok_current, and verify_norm_ok_current. Used to justify signing-oracle transcripts.
signing_attempt_sample_of_pair_currentE, signing_attempt_state_of_sample_pair_currentE	571–591. Machine-checked facts.	The structural sample pair built from the current coins reconstructs the same attempt as signing_attempt_state_of_coins.	Definitional rewrites using HAETAE_Distributions.ec. Used to identify structural attempts with current signing attempts.
structural_signing_sample_pair_sampler_ok, bounded_unit_signing_sample_pair_sampler_ok, checked_hyperball_signing_sample_pair_sampler_ok, exact_hyperball_signing_sample_pair_sampler_ok	593–623. Machine-checked facts relying on imports.	Each named sample-pair sampler satisfies the generic sampler-ok interface.	Imports distribution-ok lemmas from HAETAE_Distributions.ec. Used to derive attempt-distribution losslessness and support properties.
dsigning_attempt_state_from_sample_pair_sampler_lossless, dsigning_attempt_state_from_sample_pair_sampler_sample_ok	625–676. Machine-checked facts with conditional premise.	If a supplied sample-pair sampler is ok, the lifted attempt distribution is lossless and supported on well-formed bounded samples.	Premise: signing_sample_pair_sampler_ok. Imports dlet_ll, dmap_ll, supp_dlet, supp_dmap. Used by all sampler variants in HopGames.
dstructural_*lossless, dbounded_*lossless, dchecked_*lossless, dexact_*lossless, dbounded_*sample_ok, dchecked_*sample_ok, dexact_*sample_ok	678–747. Machine-checked facts with imported sampler facts.	Instantiates the generic lifting lemmas for structural, bounded-unit, checked-hyperball, and exact-hyperball attempt distributions.	Imports sampler-ok lemmas above and HAETAE_Distributions.ec. Used as side conditions in probabilistic hop proofs.

dexact_hyperball_signing_attempt_state_supportP, dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1, dexact_hyperball_signing_attempt_state_mu_pullback	749–799. Machine-checked facts with model boundary.	Characterizes exact-hyperball attempt support, bounds any event by one, and rewrites the exact-hyperball attempt distribution as the underlying dlet.	Imports exact_hyperball_signing_sample_pair_supportP and mu_bounded. Used by all exact-hyperball abort-zero proofs.
signing_attempt_state_reference_rejection_aborts_componentE, dexact_hyperball_signing_attempt_state_reference_rejection_abort_split, dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound	801–850. Machine-checked facts with imported probability facts.	Decomposes reference rejection aborts into reject1/reject2/pack/hint events and upper-bounds their probability by the sum of component probabilities.	Imports predU, mu_eq, mu_or, mu_bounded. Used when charging rejection components in hop bounds.
signing_attempt_state_of_sample_pair_signatureE, signing_attempt_state_of_sample_pair_pack_accepts, signing_attempt_state_of_sample_pair_pack_no_abort, dexact_hyperball_signing_attempt_state_pack_abort_zero	852–902. Machine-checked facts.	Sample-pair attempts use the current sign_internal signature, so pack accepts and exact-hyperball pack abort has probability zero.	Imports valid_signature_sign_internal, mode_crypto_bytes_gt0, support characterization, and mu0/mu_le. Used to remove the pack component of rejection loss.
dstructural_signing_attempt_state_from_samplerE, dstructural_signing_attempt_stateE, dstructural_signing_attempt_state_from_sampler_coin_sample_pairE, dexact_hyperball_signing_attempt_state_coin_sample_pairE	904–964. Machine-checked facts with imported distribution functor laws.	Equates alternate distribution presentations so loss lemmas can move between coin/sample-pair and full-state distributions.	Imports dmap_dunit, dlet_dunit, dmap_dlet, dmap_comp. Used by the loss-obligation propagation chain.
mu_dlet_le_add_all	966–1002. Machine-checked imported-library style fact.	Lifts a per-branch additive probability inequality through dlet.	Premises: $\epsilon \geq 0$ and per-branch inequality. Imports RealSeries.ler_sum, summability lemmas, dletE, weightE. Used to lift sample-pair loss through signing coins.

structural_to_exact_hyperball_sample_pair_loss_from_point_loss, structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss, structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss	1004–1053. Machine-checked implications from conditional premises.	Derives sample-pair, coin-sample-pair, and attempt-state loss obligations from the preceding obligation in the chain.	Conditional until O_{point} is supplied. Used by HopGames to charge the structural-to-exact-hyperball transition.
signing_attempt_state_of_coins_*E, signing_attempt_state_of_coins_*consistent, signing_attempt_state_of_coins_field_accepts, signing_attempt_state_of_coins_accepts_current	1055–1270. Machine-checked facts.	Every projected field of a current-coins attempt equals the corresponding deterministic signing field; therefore current attempts satisfy field and norm acceptance predicates.	Imports sign_internal, commitment_*, signature_token, valid_signature_sign_internal, and norm lemmas. Used for current signing soundness and zero-abort probabilities.
signing_attempt_reject1_bound_ge_response_bound, signing_attempt_state_reject1_accepts_from_response_norm_ok	1272–1287. Machine-checked facts.	Response-norm acceptance implies reject1 acceptance because the reject1 bound dominates the response bound.	Imports mode constants from HAETAE_Params.ec. Used in current and sample-pair reference-rejection acceptance proofs.
signing_attempt_state_balance_norm_sq_concreteE, signing_attempt_state_reject2_under_bound_concreteE, signing_attempt_state_reject2_aborts_concreteE, signing_attempt_state_reject2_accepts_from_branch_clear	1289–1330. Machine-checked facts.	Unfolds reject2 state predicates and proves that clearing the branch bit implies reject2 acceptance.	Definitional dependencies only. Used by current and sample-pair reject2 no-abort lemmas.
signing_attempt_state_of_sample_pair_reject2_branch_clear, signing_attempt_state_of_sample_pair_reject2_accepts, signing_attempt_state_of_sample_pair_reject2_no_abort, dexact_hyperball_signing_attempt_state_reject2_abort_zero	1332–1382. Machine-checked facts with model boundary.	Because sign_internal writes zero in the branch byte, sample-pair states do not reject2-abort; exact-hyperball reject2 abort has probability zero.	Imports exact-hyperball support and probability zero pattern. Used to remove the reject2 component from exact-hyperball rejection loss.
dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound, dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound, dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero	1384–1513. Machine-checked facts with model boundary.	Successively removes reject2, pack, reject1, and hint abort components until reference rejection abort probability is zero.	Imports component zero lemmas and mu_bounded. Used by HopGames when the exact-hyperball formal carrier is in force.

signing_attempt_state_of_sample_pair_response_norm_accepts, signing_attempt_state_of_sample_pair_reject1_accepts, signing_attempt_state_of_sample_pair_reject1_no_abort, dexact_hyperball_signing_attempt_state_reject1_abort_zero, signing_attempt_state_of_sample_pair_hint_norm_accepts, signing_attempt_state_of_sample_pair_hint_norm_no_abort, dexact_hyperball_signing_attempt_state_hint_norm_abort_zero	1401–1500. Machine-checked facts.	Sample-pair attempts inherit response and hint norm acceptance from current sign_internal; exact-hyperball reject1 and hint aborts are zero.	Imports response_norm_ok_current, verify_norm_ok_current, and support-to-zero pattern. Used in the reference-rejection zero chain.
signing_attempt_state_of_coins_reject2_*, signing_attempt_state_of_coins_balance_norm_sq_current, signing_attempt_state_of_coins_reject2_accepts_current	1515–1587. Machine-checked facts.	Current-coins attempts have clear reject2 branch and the expected concrete balance norm decomposition.	Imports projection equalities and branch-byte zero fact. Used by current reference-rejection acceptance.
signing_attempt_state_pack_accepts_from_valid, signing_attempt_state_reference_rejection_accepts_from_accepts, signing_attempt_state_of_coins_reference_rejection_accepts_current, signing_attempt_state_of_coins_verification_accepts_current, signing_attempt_state_of_coins_rejection_accepts_current, signing_attempt_state_of_coins_rejection_no_abort_current, signing_attempt_state_of_coins_no_abort_current	1589–1672. Machine-checked facts.	Current-coins attempts pass pack, reference rejection, verification, full rejection, and coarse abort predicates.	Imports current field acceptance, norm acceptance, reject2 acceptance, and mode_crypto_bytes_gt0. Used by current distribution zero-abort probability lemmas.
signing_attempt_state_distribution_lossless, signing_attempt_state_distribution_token_spread, signing_attempt_state_distribution_reference_rejection_abort_zero_current, signing_attempt_state_distribution_rejection_abort_zero_current	1674–1724. Machine-checked facts.	The current attempt distribution is lossless, preserves the signing-token point bound, and has zero current reference/full rejection abort probability.	Imports signing_coin_distribution_lossless, signing_coin_distribution_token_point_bound, dmapE, and mu0. Used by sampler prequery and current-signing hop steps.

sign_internal_simulation_relation, signing_attempt_transcript_valid, signing_attempt_transcript_relation, sign_internal_transcript_relation, sign_internal_record_relation	1726–1790. Machine-checked facts.	Turns honest formal signing into simulation and transcript/record relations.	Imports transcript_of_signature, transcript_valid, and verify_internal_sign_internal. Used by transcript logging games and NMA lifts.
signing_record_relation_sound, signing_record_log_sound_cons, signing_record_log_sound_transcripts_valid, signing_record_queries_cons, signing_record_transcripts_cons	1792–1838. Machine-checked facts.	Sound record logs imply simulated signatures and valid transcript lists; cons lemmas expose list structure.	Imports list induction and transcript predicates. Used by HopGames transcript log invariants and query-budget bookkeeping.
rejection_bound_parts_nonnegative, rejection_sampling_bound_obligation_holds	1840–1854. Machine-checked facts relying on imports.	Shows the named rejection/Fiat-Shamir-with-aborts loss terms are nonnegative.	Imports nonnegativity lemmas from HAETAE_Reductions.ec. This proves nonnegativity only; smallness and cryptographic interpretation remain outside this file.
signing_attempt_state_of_sample_pair_*E, signing_attempt_state_of_sample_pair_*consistent, signing_attempt_state_of_sample_pair_field_accepts, signing_attempt_state_of_sample_pair_valid_signature_accepts, signing_attempt_state_of_sample_pair_accepts_current	1856–2073. Machine-checked facts.	Sample-pair attempts have the same deterministic signature fields as the current signer and therefore satisfy field, validity, response, and hint acceptance.	Imports projection equalities, sign_internal, formal correctness, and norm lemmas. Used by exact-hyperball support-to-acceptance proofs.
signing_attempt_state_of_sample_pair_verification_accepts_current, signing_attempt_state_of_sample_pair_reference_rejection_accepts, signing_attempt_state_of_sample_pair_reference_rejection_no_abort, signing_attempt_state_of_sample_pair_rejection_accepts_current, signing_attempt_state_of_sample_pair_rejection_no_abort_current	2075–2130. Machine-checked facts.	Sample-pair states pass verification, reference rejection, full rejection, and the corresponding no-abort predicates.	Imports field acceptance, reference-rejection acceptance, and deterministic signer facts. Used directly by exact-hyperball no-abort lemmas.

dexact_hyperball_signing_attempt_state_reference_rejection_accepts, dexact_hyperball_signing_attempt_state_reference_rejection_no_abort, dexact_hyperball_signing_attempt_state_rejection_accepts, dexact_hyperball_signing_attempt_state_rejection_no_abort, dexact_hyperball_signing_attempt_state_rejection_abort_zero	2132–2202. Machine-checked facts with model boundary.	Any state in the current exact-hyperball attempt distribution passes reference and full rejection; full rejection abort probability is zero.	Imports support characterization and sample-pair acceptance lemmas. Used by exact-hyperball hop games. This remains a formal-carrier theorem, not a paper-sampler correspondence theorem.

1.28 Script companion coverage for HAETAE ROM

HAETAE_ROM.ec instantiates the generic random-oracle library with HAETAE-specific query and output types. This file is small but central: all ROM programming, counted-query, and sampler-prequery events refer to its constructors.

Script object family	Mathematical reading	Downstream role
ro_query constructors	MessageHashQuery, ChallengeHashQuery, MatrixExpandQuery, and SamplerExpandQuery partition random-oracle inputs by protocol role.	Bad-event flags and query-budget predicates count only the relevant constructor families, especially challenge and sampler queries.
ro_output tuple	The random-oracle output is a tuple crh × challenge × matrix × random_coins. Projection functions select the component appropriate to the query.	HAETAE_Scheme.ec uses ro_message_hash, ro_challenge_hash, ro_matrix, and ro_signing_coins.
dro_output	The output distribution dispatches by query kind: message hash samples dcrh, challenge hash samples dchallenge, matrix expansion samples dmatrix, and sampler expansion samples drandom_coins.	The FullRO clone uses this as the oracle's output law. Losslessness is needed for game proofs that call the oracle.
cloneimportFullROasHAETAE_RO	The generic full random oracle is instantiated with HAETAE query/output types and dro_output.	HopGames and ROM-programming modules reason about HAETAE_RO.FRO.m and oracle methods through this clone.

sampler_expand_query_ dro_output_ro_signing_ coins	For sampler-expansion queries, projecting ro_signing_coins from dro_ output has the same law as drandom_ coins.	Used by sampler-freshness lemmas in HAETAE_HopGames.ec to replace a fresh ROM sampler query by the signing-coin distribution.

Boundary: this is a random-oracle interface definition, not a theorem that any concrete SHAKE invocation behaves randomly. That modeling step remains the ROM assumption boundary.

1.29 Script companion coverage for HAETAE Scheme

HAETAE_Scheme.ec clones the generic signature-game interface and defines the scheme procedures used by HAETAE_Security.ec. It is the interface contract between the algebraic model and generic EUF-CMA/NMA games.

Script object family	Mathematical reading	Downstream role
cloneimportSig_ ROMasSIG	Instantiates generic signature-game types with HAETAE carriers and the HAETAE random-oracle output distribution.	All SIG.EUF_CMA, SIG.UF_NMA, adversary, and scheme module types used in Security and HopGames come from this clone.
haetae_mode	An abstract operation selecting the parameter mode for the scheme instance.	Final theorems are for the formal scheme at this mode; later premises such as fs_with_ aborts_interfaces_ readyhaetae_mode are mode-indexed.
HAETAE(H).kg	Samples sd from dseed, derives rhoprime, queries the ROM at matrix_expand_queryhaetae_ moderhoprime, and returns keygen_ internalhaetae_moderhoprime.	Defines the key-generation behavior used by correctness and all security games.
HAETAE(H).sign	Samples drandom_coins, queries sampler_expand_query, derives the public key from the secret key, queries message hash and challenge hash, and returns sign_internal.	Fixes the query order and query kinds used by ROM programming and bad-event proofs. The signer ignores the challenge output except through the formal sign_internal model.

HAETAE(H).verify	Returns verify_internalhaetae_modepkmctxsig.	Correctness and EUF-CMA success events use this predicate.

Boundary: this file exposes the formal scheme to generic games. It does not prove security; it also does not prove byte-level implementation equivalence. The checked facts about validity and correctness are imported from HAETAE_Algebra.ec and consumed later in HAETAE_Security.ec.

1.30 Script companion coverage for HAETAE Events and hash support

Three smaller manifest files complete the next dependency layer: HAETAE_Events.ec, HAETAE_FIPS202.ec, and HAETAE_Keccak1600.ec. They are not final security files, but they matter because the proof imports their definitions and facts.

File	Definitions and checked lemma families	Boundary and downstream role
HAETAE_Events.ec	Defines keygen_rejection_failure, signing_rejection_failure, verification_rejection_failure, and any_rejection_failure. Lemmas such as keygen_rejection_free_current, signing_rejection_free_current, verification_rejection_free_current, and accepted_transcript_rejection_free_current prove that the current formal signer avoids these event predicates.	Machine-checked event vocabulary. Used by rejection reductions and by the security proof's rejection-free branches. It does not estimate rejection probabilities for a separate implementation.
HAETAE_FIPS202.ec	Defines byte-list fips202_state, SHAKE256 constants, the one-block absorb/padding model, shake256_squeeze, and shake256. Lemmas prove constant equalities and output-size facts.	Imported by HAETAE_Algebra.ec to model seed derivation. It is checked hash plumbing, not a ROM proof and not a full FIPS validation certificate.
HAETAE_Keccak1600.ec	Defines lanes, byte/lane conversion, rho offsets, round constants, theta, rho-pi, chi, iota, keccak_round, keccak_f1600_lanes, and keccak_f1600_bytes. Lemmas prove sizes, bit-level unfoldings, and round-function projection equations.	Imported by HAETAE_FIPS202.ec. These are machine-checked definitions and local consistency lemmas for the formal byte-list permutation. They do not turn the final theorem into a standard-model SHAKE theorem.

For a reimplementation, these support files should be handled before claiming the algebra layer is reproduced. Their imported facts are dependencies of the formal model, even though the top-level security theorem remains a random-oracle-model theorem.

1.31 Evidence required for a standalone claim

If a future version of the documentation is advertised as sufficient by itself, it must contain enough evidence for an independent reader to rebuild the proof without opening the repository. At minimum, it must satisfy the following requirements.

Required evidence	**Why it is necessary**	**Present here?**
Literal EasyCrypt source for every manifest file	EasyCrypt checks exact syntax, definitions, imports, and proof scripts, not prose descriptions.	No
Exact module restriction sets	Security games rely on separation between adversaries, oracles, simulators, and hidden game state.	Representative only
Full helper-lemma statements and proofs	Top-level theorems depend on local facts whose names alone do not determine their statements or proof obligations.	No
Complete import and library version contract	Proof success can depend on library names, theories, notations, and solver behavior.	Partial
Traceability matrix from prose to checked declarations	A reader needs to know which checked object discharges each mathematical step.	Required, not included in full
Transcript of a successful verification run	The claim is machine-checked only after the checker has accepted the complete source tree.	Build command only

This table is an acceptance gate. The current document intentionally leaves several entries as "No" or "Partial", so it must not be cited as a self-contained proof archive.

1.32 What this report deliberately does not claim

The checked development is precise, but its scope is narrow. Reimplementing the proof from this report does not prove any of the following.

- A standard-model theorem replacing the random oracle by SHAKE.
- Concrete MLWE or Module-SIS bit-security estimates for HAETAE parameter sets.
- Constant-time implementation security.
- Byte-level parsing correctness for all concrete encodings.
- A machine-checked equivalence theorem

$$\mathsf{Spec}_{260204} \equiv \mathsf{ECModel}.$$

- A cost-semantics theorem bounding the running time of the reductions.

Those are boundary obligations. The report explains where each boundary enters so that a reimplementation does not silently promote the checked theorem into a stronger end-to-end certification claim.

1.33 Repository paths

The proof gate lives under the project directory `haetae-security`. The active proof-management surface is:

```
provable-security/
  proof-files.txt
  verify-provable-security.sh
  easycrypt/
  logs/
```

The manifest source files also exist at the project root. The current proof gate compiles the copies under `provable-security/easycrypt/`. A reimplementation may keep only one copy, but then the verification script and import paths must be adjusted consistently.

Chapter 2

Verification Workflow

2.1 Prerequisites

Install an EasyCrypt version capable of compiling the current scripts and make sure the command is visible in the shell:

```
easycrypt --version
```

The proof also imports supporting EasyCrypt libraries from `kyber-security`. A fresh reimplementation should preserve this include path until all dependencies have been audited and replaced.

2.2 The manifest

The manifest is the list of files that define the proof gate. It should contain exactly these entries, in dependency-compatible order:

```
HAETAE_Security.ec
Sig_ROM.ec
HAETAE_HopGames.ec
HAETAE_Reductions.ec
HAETAE_ROM.ec
HAETAE_ROM_Programming.ec
HAETAE_Scheme.ec
HAETAE_Transcript.ec
HAETAE_Events.ec
HAETAE_Assumptions.ec
HAETAE_Distributions.ec
HAETAE_Rejection.ec
HAETAE_Algebra.ec
HAETAE_Params.ec
HAETAE_FIPS202.ec
HAETAE_Keccak1600.ec
```

Generated known-answer-test files, archival experiments, and unrelated implementation validation artifacts should not be added to this gate merely because they compile. The manifest is a proof-surface boundary, not a list of all EasyCrypt files in the repository.

2.3 Running the proof gate

From the project root, run:

```
sh provable-security/verify-provable-security.sh
```

The script compiles each manifest entry with include paths equivalent to:

```
easycrypt compile provable-security/easycrypt/<file>.ec \
  -I provable-security/easycrypt \
  -I kyber-security
```

The run is successful only if the final line is:

```
RESULT: PASS
```

2.4 Independent audit commands

A reimplementation should also provide small audit commands. The following checks are not substitutes for EasyCrypt, but they catch common proof-surface drift.

```
# Count declarations and confirm that the inventory is stable.
./dissertation/etc/count-easycrypt-declarations.sh

# Confirm that no local proof file uses unchecked proof holes.
rg -n "\b(axiom|admit\.|sorry\.)\b" provable-security/easycrypt *.ec

# Confirm that manifest copies match root copies, if both are retained.
rg -v "^#|^$" provable-security/proof-files.txt |
while read f; do
  cmp -s "$f" "provable-security/easycrypt/$f" || echo "$f␣differs"
done
```

For the current artifact, the declaration inventory is:

```
TOTAL,22046,63,678,113,1088,0,105
```

The columns are file lines, type declarations, operation declarations, module declarations, lemma declarations, axiom declarations, and Hoare/equivalence judgments.

Chapter 3

Mathematical Theorem Boundary

3.1 Security game

The informal target is the random-oracle EUF-CMA experiment. An adversary A receives a public key, random-oracle access, and signing-oracle access. It outputs $(m^\star, ctx^\star, \sigma^\star)$. It wins when

$$\mathsf{Verify}^H(pk, m^\star, ctx^\star, \sigma^\star) = 1$$

and $(m^\star, ctx^\star)$ was not previously asked of the signing oracle. The formal checked advantage is the success probability of

`SIG.EUF_CMA(H,HAETAE,A).main`.

3.2 Final checked implication

The strongest checked theorem emphasized by the current proof surface is the following EasyCrypt implication.

Theorem 3.1 (Checked counted-ROM implication, informal rendering)**.** *For any EasyCrypt memory parameter, if the final exact-hyperball NMA hop satisfies*

$$\Pr[\mathit{UF\_NMA}(\mathit{ROMInternalTranscriptPaperSimAsNMA}(A, \mathit{ROExactHyperballPaperSimSampler}))] + \mathit{rejection\_sampling\_loss\_term} \le \Pr[\mathit{MLWE\_Game}(B_{\mathsf{MLWE}})] + \Pr[\mathit{BimodalSelfTargetMSIS\_Game}(B_{\mathsf{Bimoda}}$$

and if

$$\Pr[\mathit{MLWE\_Game}(B_{\mathsf{MLWE}})] \le \mathit{mlwe\_hardness\_term},$$

and

$$\Pr[\mathit{ModuleSIS\_Game}(\mathit{BimodalMSIS\_As\_ModuleSIS}(B_{\mathsf{BimodalSelfTargetMSIS}}))] \le \mathit{module\_sis\_hardness\_term},$$

then

$$\Pr[\mathit{SIG.EUF\_CMA(H,HAETAE,A).main}] \le \mathit{haetae\_euf\_counted\_rom\_bound}.$$

The EasyCrypt theorem name is:

```
euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal
```

3.3 Why the theorem is conditional

The modules B_{MLWE} and $B_{\mathsf{BimodalSelfTargetMSIS}}$ are declared reduction modules in the final theorem surface. The top-level composition lemma does not by itself construct these solvers from A. It composes explicitly stated probability premises. A paper proof may describe constructed reductions, but a faithful description of this checked theorem must say that the final composition consumes declared reduction modules and antecedents.

3.4 The counted-ROM bound

The reduction file defines:

```
op mlwe_hardness_term : real.
op module_sis_hardness_term : real.
op bimodal_to_msis_reduction_loss_term : real = 0%r.

op signature_query_count : real = 2%r.
op hash_query_count : real = 2%r.
op challenge_support_cardinality_lower_bound : real =
  288230376151711744%r.

op counted_rom_collision_term =
  (rom_total_query_budget * rom_total_query_budget) /
    challenge_support_cardinality_lower_bound.

op counted_rom_prequery_term =
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /
    challenge_support_cardinality_lower_bound.

op counted_rom_min_entropy_term =
  (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) /
    challenge_support_cardinality_lower_bound.
```

Thus

$$q_H = 2, \qquad q_S = 2, \qquad N = 288230376151711744 = 2^{58}.$$

The ROM loss is

$$\frac{(2+2+1)^2}{2^{58}} + \frac{2(2+1)}{2^{58}} + \frac{2(2+1)}{2^{58}} = \frac{25+6+6}{2^{58}} = \frac{37}{2^{58}}.$$

3.5 The vacuity warning

The proof also defines a paper-style rejection/Fiat-Shamir-with-aborts loss:

```
op rejection_sampling_loss_term =
  fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.
```

The checked constants imply:

```
lemma fs_with_aborts_min_entropy_termE :
  fs_with_aborts_min_entropy_term = 12%r.
```

```
lemma rejection_sampling_loss_term_ge1 :
  1%r <= rejection_sampling_loss_term.
```

Therefore a premise of the form

$$\Pr[G] + \texttt{rejection\_sampling\_loss\_term} \leq \Pr[\mathsf{MLWE}] + \Pr[\mathsf{BimodalSelfTargetMSIS}]$$

is not a useful concrete-security premise in the current constants. The right side is at most 2, while the extra loss term is already at least 1 and has a min-entropy component equal to 12. The checked theorem is still a valid logical implication, but it should be cited with this warning attached.

Chapter 4

Pen-and-Paper Reimplementation

4.1 Proof skeleton

A paper proof reconstructed from the checked files should be written as the following chain.

1. Define the real EUF-CMA game G_0.
2. Replace real signing by a simulator while preserving the visible distribution or charging an explicit signing-hop loss.
3. Expose transcript state so that commitments, challenges, responses, and oracle queries become explicit.
4. Move from adaptive signing access to an NMA-style internal transcript surface.
5. Program the random oracle and charge collision, prequery, and min-entropy bad events.
6. Replace structural or paper sampling views by the exact hyperball sampler used by the formal model.
7. Apply special soundness to compatible accepting transcript pairs.
8. Convert the extracted bimodal self-target relation to the Module-SIS hardness game.
9. Compose MLWE, Module-SIS, bimodal-to-MSIS, and counted-ROM terms.

Every step must be labelled either as an exact equivalence, a bad-event bound, a sampling-distance bound, a reduction step, or a real-arithmetic normalization. Do not add rejection, exact-sampling, or active-signing losses as final summands if the EasyCrypt theorem already internalizes them into hop premises. Doing so double-counts proof obligations.

4.2 Game notation

Use the following names in the paper proof.

Paper object	**EasyCrypt counterpart**
Real EUF-CMA game	`SIG.EUF_CMA(H,HAETAE,A).main`.

Paper object	EasyCrypt counterpart
No-message game	`SIG.UF_NMA(H,HAETAE,A).main`.
Random oracle	`HAETAE_RO.FRO` cloned from the generic full random oracle.
Signing oracle	Modules named `O.sign` inside transcript and budgeted NMA games.
MLWE reduction game	`MLWE_Game(Bmlwe).main`.
Bimodal self-target MSIS game	`BimodalSelfTargetMSIS_Game(Bbimodal).main`.
Module-SIS game	`ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal) ).main`.

4.3 Bad events

The random-oracle and sampler bad events should be separated.

- The counted-ROM collision term accounts for collisions among programmed or observed ROM sites.
- The counted-ROM prequery term accounts for programmed challenge sites that were already queried in the counted transcript ledger.
- The counted-ROM min-entropy term accounts for the challenge entropy bad event in the counted transcript ledger.
- The sampler prequery event `sampler_bad_prequery` is a local active-signing event. It is bounded by a different expression involving the adversary hash log and the sampler self-log.

The paper proof must not identify `sampler_bad_prequery` with `counted_rom_prequery_term`. They are related proof ideas, but they are different events in the checked source.

4.4 Special soundness and extraction

The extraction step is not a conclusion from a single valid transcript. It requires a forking pair: two accepting transcripts sharing the appropriate public data and carrying distinct challenges. The paper proof should state the obligation as:

$$\mathsf{ValidForkingPair}(tr_1, tr_2) \Longrightarrow \mathsf{Solve}_{\mathsf{BimodalSelfTargetMSIS/Module\text{-}SIS}}(tr_1, tr_2).$$

The checked anchors are:

```
extract_bimodal_forking_some
extract_bimodal_forking_nonempty
valid_forking_pair_public_equations
bimodal_to_module_sis_valid
bimodal_solver_lift_exact
bimodal_mode_solver_lifted_distribution_exact
bimodal_msis_as_module_sis_exact
```

Chapter 5

EasyCrypt Reimplementation Plan

5.1 Development order

Reimplement the files from leaves to root. Do not begin with the final theorem. Use the following order.

1. `HAETAE_Params.ec`
2. `HAETAE_Keccak1600.ec`
3. `HAETAE_FIPS202.ec`
4. `HAETAE_Algebra.ec`
5. `HAETAE_Distributions.ec`
6. `HAETAE_Events.ec`
7. `HAETAE_Assumptions.ec`
8. `Sig_ROM.ec`
9. `HAETAE_ROM.ec`
10. `HAETAE_Scheme.ec`
11. `HAETAE_Transcript.ec`
12. `HAETAE_Reductions.ec`
13. `HAETAE_Rejection.ec`
14. `HAETAE_ROM_Programming.ec`
15. `HAETAE_HopGames.ec`
16. `HAETAE_Security.ec`

The manifest compiles root-first because each file is checked independently with imports resolved by EasyCrypt. Human reimplementation should proceed leaf-first because each later file uses definitions from earlier ones.

5.2 Global proof idioms

Use four recurring EasyCrypt idioms.

Typed interfaces. Use module types for adversaries, schemes, oracles, and hardness solvers. This prevents games from depending on implementation details that should remain abstract.

State exposure. When a paper proof says "the simulator records queries", make a module with explicit state variables for logs, counters, and bad flags.

Local lemmas before global theorem. First prove preservation lemmas, support lemmas, losslessness facts, and pointwise arithmetic bounds. Then assemble them in top-level security lemmas.

No silent axioms. Do not use `axiom`, `admit.`, or `sorry.`. If a fact is an external assumption, represent it as a declared operation or module interface and make the final theorem conditional on it.

5.3 Placeholder convention

The snippets in the next chapter are intentionally schematic where the full source would obscure the proof architecture. In particular, code fragments containing `...` are not EasyCrypt programs. They mark locations where the complete project source must be consulted or rederived. This convention is a guardrail against a false kind of self-containment: copying the snippets alone will not produce a compiling proof, and the report should not be cited as a literal replacement for the checked `.ec` files.

For a genuinely standalone manual, every occurrence of `...` would have to be expanded into exact EasyCrypt source, every helper lemma would need its full statement and proof, and every module declaration would need its exact restriction set. That is a different artifact from this roadmap.

Before claiming that a documentation-derived proof tree is machine checked, run a placeholder audit over the generated files. The audit is deliberately simple: any remaining ellipsis or proof-hole marker must be explained as a comment or removed from the proof surface.

```
rg -n "\.\.\.|TODO|FIXME|admit\.|sorry\.|\baxiom\b" \
  provable-security/easycrypt
```

The expected result for the active proof surface is no unchecked proof hole. A hit on `...` in executable EasyCrypt means the roadmap has not yet been expanded into source.

Chapter 6

File-by-File Blueprint

6.1 HAETAE_Params.ec

Define the fixed HAETAE parameter vocabulary.

```
type mode = [ Mode2 | Mode3 | Mode5 ].

op n : int = 256.
op q : int = 64513.
op dq : int = 2 * q.

op mode_k (md : mode) : int = ...
op mode_l (md : mode) : int = ...
op mode_tau (md : mode) : int = ...
```

Required lemmas:

- positivity of byte sizes, n, q, and all mode-dependent bounds;
- $dq = 2q = 129026$;
- $\tau \leq n$ for every mode;
- positivity of public-key and signature-size constants.

These lemmas are simple case splits, but they are essential because later probability and support facts need positive denominators and nonempty domains.

6.2 HAETAE_Keccak1600.ec and HAETAE_FIPS202.ec

These files provide modeled hash-interface support. They do not replace the random oracle in the security theorem. A reimplementation should define:

- byte and state list types;
- state-size, rate, capacity, domain-separator, and padding constants;
- deterministic absorb, squeeze, and permutation operations;
- length and well-formedness lemmas needed by the algebra layer.

Keep the cryptographic boundary explicit: compiling these files does not prove standard-model SHAKE security.

6.3 HAETAE_Algebra.ec

This is the algebraic model of HAETAE objects. Define carrier types for:

- bytes, seeds, CRHs, random coins, messages, and contexts;
- coefficients, polynomials, challenges, polynomial vectors, and matrices;
- public keys, secret keys, and signatures.

Then define total operations for:

- modular coefficient arithmetic;
- polynomial and vector operations;
- key generation internals;
- signing internals;
- verification internals;
- transcript commitment and challenge components;
- norm and validity predicates.

The most important final facts are:

```
lemma verify_internal_sign_internal ...
lemma valid_signature_sign_internal ...
lemma secret_norm_ok_current ...
lemma response_norm_ok_current ...
lemma verify_norm_ok_current ...
```

The proof model stores squared norm thresholds. The validity reading is:

$$\|z\|^2 \le B^2 \quad \text{implemented as} \quad \texttt{normsq}(z) \le \texttt{mode\_b*sq}(md).$$

Do not replace this by a strict inequality unless the corresponding rejection predicate actually uses a strict bound.

6.4 HAETAE_Distributions.ec

Define the distributions used by key generation, signing, challenges, and structural samplers.

Essential distribution properties:

- losslessness of byte, seed, CRH, random-coin, coefficient, polynomial, vector, matrix, and challenge distributions;
- support facts showing sampled objects satisfy the predicates expected by the algebra layer;

- projection facts connecting structural signing samples to response, challenge, and randomness components.

Every sampler used by a program that should terminate with probability one must have a corresponding losslessness lemma or a local proof of losslessness.

6.5 HAETAE_Events.ec

Define named event predicates rather than spelling conditions repeatedly. Events should include rejection, bad prequery, bad min-entropy, collision, and validity conditions needed by the game hops.

This file should contain no heavy proof. Its job is vocabulary alignment: later files can reason about the same event name instead of duplicating Boolean formulas.

6.6 HAETAE_Assumptions.ec

Introduce the external hardness games and reduction interfaces.

```
module type MLWE_Reduction = {
  proc main() : bool
}.

module type BimodalSelfTargetMSIS_Reduction = {
  proc main() : bool
}.

module type ModuleSIS_Reduction = {
  proc main() : bool
}.
```

Define game wrappers that call these modules:

```
module MLWE_Game(B : MLWE_Reduction) = {
  proc main() : bool = {
    var b : bool;
    b <@ B.main();
    return b;
  }
}.
```

Also define the bimodal-to-Module-SIS adapter and prove exactness lemmas:

```
bimodal_to_module_sis_valid
bimodal_solver_lift_exact
bimodal_mode_solver_lifted_distribution_exact
bimodal_msis_as_module_sis_exact
```

These are not concrete cryptanalytic estimates. They are formal assumption surfaces used by the final reduction theorem.

6.7 Sig_ROM.ec

Define the generic random-oracle signature game independent of HAETAE.

```
type pkey.
type skey.
type message.
type context.
type signature.
type ro_query.
type ro_output.

op [lossless] dro_output : ro_query -> ro_output distr.
```

Define module types:

```
module type Oracle = { include FRO [init, get] }.
module type POracle = { include FRO [get] }.
module type Scheme(O : POracle) = { ... }.
module type SignOracle = { proc sign(m : message, ctx : context) : signature }.
module type Adversary(H : POracle, O : SignOracle) = { ... }.
module type NMA_Adversary(H : POracle) = { ... }.
```

Define:

- correctness game;
- EUF-CMA game;
- UF-NMA game;
- NMA-as-CMA adapter;
- freshness predicates for signed message-context pairs.

The generic game should not contain HAETAE-specific facts. It must be reusable.

6.8 HAETAE_ROM.ec

Instantiate random-oracle query and output types for HAETAE. Use typed query constructors for distinct hash domains:

- matrix expansion;
- message hashing;
- sampler expansion;
- challenge hashing.

The reason for typed constructors is domain separation. A proof should not be able to confuse a sampler-expansion query with a challenge query.

6.9 HAETAE_Scheme.ec

Clone the generic signature interface and instantiate it with HAETAE types and the HAETAE random-oracle output distribution:

```
clone import Sig_ROM as SIG with
  type pkey <- pkey,
  type skey <- skey,
  ...
  op dro_output <- dro_output.
```

Define the formal scheme module:

```
module HAETAE(H : SIG.POracle) = {
  proc kg() : pkey * skey = { ... }
  proc sign(sk : skey, m : message, ctx : context) : signature = { ... }
  proc verify(pk : pkey, m : message, ctx : context,
              sig : signature) : bool = { ... }
}.
```

The current wrapper makes ROM calls visible around deterministic internal operations. In particular, signing samples typed random coins, asks the sampler ROM query, interprets the output as signing coins, computes message and challenge ROM queries, and finally calls `sign_internal`.

6.10 HAETAE_Transcript.ec

Define transcript records and validity predicates. A transcript should contain at least the public key, message, context, signature, commitment components, challenge query, and challenge value needed by extraction.

Key proof obligations:

```
transcript_valid_signature_challenge_query
transcript_valid_signature_challenge_matches
transcript_valid_signature_challengeE
extract_bimodal_forking_some
extract_bimodal_forking_nonempty
valid_forking_pair_public_equations
```

The extraction lemmas should require a valid forking pair. Do not prove or state special soundness from one accepting transcript alone.

6.11 HAETAE_Reductions.ec

Define all real-valued loss terms and prove their arithmetic properties.

Required groups:

- abstract hardness terms: `mlwe_hardness_term`, `module_sis_hardness_term`;
- bimodal-to-MSIS loss slot, currently zero;
- fixed query counts $q_H = q_S = 2$;

- challenge support lower bound $N = 2^{58}$;
- counted-ROM collision, prequery, and min-entropy terms;
- paper-style rejection/Fiat-Shamir-with-aborts terms;
- grouped final bounds `haetae_euf_bound` and `haetae_euf_counted_rom_bound`.

Arithmetic lemmas to reprove:

```
haetae_euf_counted_rom_bound_groupedE
counted_rom_collision_termE
counted_rom_prequery_termE
counted_rom_min_entropy_termE
counted_rom_programming_loss_termE
counted_rom_programming_loss_term_concreteE
counted_rom_programming_loss_term_subunit
fs_with_aborts_min_entropy_termE
rejection_sampling_loss_term_ge1
bimodal_to_msis_reduction_loss_termE
```

This file is where most accidental theorem inflation can be caught. The final counted-ROM bound is not the same as the older paper-style rejection bound.

6.12 HAETAE_Rejection.ec

Define signing-attempt state, rejection predicates, and the relationship between reference rejection and the modeled signing state.

Important predicates:

- `signing_attempt_state_valid_signature_accepts`;
- `signing_attempt_state_reject1_accepts`;
- `signing_attempt_state_reject2_accepts`;
- `signing_attempt_state_reference_rejection_accepts`;
- `signing_attempt_state_rejection_accepts`.

Important obligations:

- bound rejection probabilities by named terms;
- show exact-hyperball signing attempt states satisfy the reference rejection predicates;
- prove zero-abort facts for the exact paths currently used by the model;
- expose coarse fallback bounds honestly when they are used.

6.13 HAETAE_ROM_Programming.ec

Define the counted random-oracle programming surface.

This file should model:

- lazy ROM maps;
- observed query logs;
- programmed query sites;
- collision and prequery bad events;
- preservation of good state under oracle operations;
- probability bounds for programming failures.

Use explicit counters and support lower bounds. Do not write a prose "fresh with high probability" argument without a program variable tracking the queries that make the statement meaningful.

6.14 HAETAE_HopGames.ec

This is the largest file because it contains the concrete hybrid machinery. Reimplement it in layers.

Layer 1: concrete lazy-ROM laws

Prove sampler expansion freshness:

```
concrete_lazy_rom_sampler_expand_query_fresh_le
concrete_lazy_rom_sampler_expand_query_fresh_eq
```

Mathematical reading: if `sampler_expand_query`(r) is not in the lazy-ROM table, then reading the ROM at that point yields a signing-coin distribution bounded by the ideal coin distribution.

Layer 2: signing-oracle clean invariants

Expose the active signing oracle state and prove that clean conditions are preserved:

```
concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh
```

The clean condition should include sampler-ROM coverage, absence of the current sampler prequery bad event, budget discipline, public-key consistency, and secret-key consistency.

Layer 3: one-call lifting

Prove the clean one-call bound:

```
concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface
```

Its mathematical shape is:

$$\begin{aligned}&\Pr[O_{\mathsf{attempt}}.\mathsf{sign} : p(res) \wedge \neg \mathsf{sampler\_bad\_prequery}] \\ &\quad \le \Pr[O_{\mathsf{exact}}.\mathsf{sign} : p(res)] + \texttt{rejection\_sampling\_loss\_term}.\end{aligned}$$

Layer 4: NMA and transcript game lifting

Lift the one-call facts through NMA-style transcript games:

```
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting

rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
```

This is where the current development still contains coarse fallback bounds. A faithful reimplementation should keep those fallbacks visible and should not describe the final result as non-vacuous.

6.15 HAETAE_Security.ec

Assemble the final theorem family.

First prove correctness:

```
op correctness_obligation = ...
lemma correctness_obligation_holds : correctness_obligation.
lemma correctness_perfect &m :
  Pr[SIG.Correctness(H, HAETAE, A).main() @ &m : res] = 0%r.
```

Then assemble progressively stronger security lemmas:

- generic EUF-CMA security from hop interfaces;
- ROM-transcript security;
- bimodal-to-Module-SIS lift;
- counted-ROM grouped bound;
- paper-sampling and exact-hyperball bridges;
- final checked counted-ROM implication.

The final theorem should look like:

```
lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m :
  Pr[SIG.UF_NMA(H, HAETAE,
      ROMInternalTranscriptPaperSimAsNMA
        (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] +
    rejection_sampling_loss_term <=
    Pr[MLWE_Game(Bmlwe).main() @ &m : res] +
    Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] =>
  Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term =>
  Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal)).main()
      @ &m : res] <= module_sis_hardness_term =>
  Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <=
    haetae_euf_counted_rom_bound.
```

The proof should be mostly composition: apply the previous bridge lemma, pass the hop premise, pass the MLWE premise, and pass the Module-SIS premise.

Chapter 7

Reimplementation Checklists

7.1 Pen-and-paper checklist

Before writing EasyCrypt, verify the paper proof has these properties.

1. The starting game is the same EUF-CMA game later encoded in `Sig_ROM.ec`.
2. The source game has no syntactic query cap; fixed counts belong to the wrapper and bound ledger.
3. Every hop is classified as exact, bad-event, sampling-distance, or reduction.
4. Random-oracle programming freshness is stated in terms of explicit query sets.
5. Sampler-prequery events are separated from counted-ROM challenge prequery events.
6. Forking/extraction uses two compatible accepting transcripts with distinct challenges.
7. MLWE and Module-SIS hardness are external assumptions represented by abstract games.
8. The final counted-ROM bound is not mixed with the older `haetae_euf_bound` rejection ledger.
9. The final theorem is marked conditional and carries the vacuity warning.

7.2 EasyCrypt checklist

Before accepting a reimplementation, run these checks.

1. The reproduction mode is declared: audit mode with companion sources, literal standalone mode with full source in the document, or independent reimplementation mode with a traceability matrix.
2. If `.ec` scripts are supplied in script companion mode, every security-relevant script block has a corresponding mathematical explanation in the report.
3. The script-companion coverage ledger is complete: no manifest file, security-critical declaration, nontrivial lemma, tactic block, module restriction, or external assumption remains without a prose explanation.
4. Every manifest file compiles.

5. There are no `axiom`, `admit.`, or `sorry.` tokens in the proof surface.
6. There are no executable `...`, `TODO`, or `FIXME` placeholders in the proof surface.
7. Each declared module in `HAETAE_Security.ec` has the right restriction set, so the adversary cannot call hidden game internals.
8. All distributions sampled by lossless procedures have losslessness lemmas.
9. All denominators in real bounds have positivity lemmas.
10. The counted-ROM concrete arithmetic rewrites to $37/2^{58}$.
11. The rejection min-entropy term rewrites to 12.
12. The final theorem consumes declared reduction modules and explicit premises rather than pretending to construct all solvers inside the final composition.
13. Every theorem, loss term, and declared module used by the final theorem is traceable either to the companion source or to a checked object in the new reimplementation.

7.3 Module restriction-set caveat

Module restriction sets are part of the proof, not formatting. They prevent an adversary or oracle module from calling game internals that should be hidden. The checklist item above therefore requires exact source-level comparison, not a verbal approximation.

For example, the final mechanized ROM-transcript/NMA section begins with declarations of this form:

```
section MechanizedROMTranscriptConstructedNMALift.

declare module H <: SIG.Oracle {-SIG.EUF_CMA,
                                -EUF_CMA_InternalTranscriptSign,
                                -EUF_CMA_ROMInternalTranscriptSign,
                                -ROMInternalTranscriptAsNMA,
                                -ROMInternalTranscriptPublicSimAsNMA,
                                -ROMInternalTranscriptPaperSimAsNMA,
                                -ROMInternalTranscriptBudgetedPaperSimAsNMA,
                                -ROMPaperSimSigningSampler,
                                -RealSigningPaperSimSampler,
                                -ROSigningAttemptPaperSimSampler,
                                -HAETAERejectionPaperSimSampler,
                                -ExactHyperballHAETAERejectionPaperSimSampler,
                                -ExactHyperballPaperSimSampler,
                                -ROExactHyperballPaperSimSampler}.
declare module A <: SIG.Adversary {-H, -HAETAE, -SIG.EUF_CMA,
                                   -EUF_CMA_InternalTranscriptSign,
                                   -EUF_CMA_ROMInternalTranscriptSign,
                                   -ROMInternalTranscriptAsNMA,
                                   -ROMInternalTranscriptPublicSimAsNMA,
                                   -ROMInternalTranscriptPaperSimAsNMA,
                                   -ROMInternalTranscriptBudgetedPaperSimAsNMA,
                                   -ROMPaperSimSigningSampler,
                                   -RealSigningPaperSimSampler,
```

```
                                  -ROSigningAttemptPaperSimSampler,
                                  -HAETAERejectionPaperSimSampler,
                                  -ExactHyperballHAETAERejectionPaperSimSampler,
                                  -ExactHyperballPaperSimSampler,
                                  -ROExactHyperballPaperSimSampler}.
declare module Samp <: PaperSimSigningSampler {-H, -A,
                                               -ROMInternalTranscriptPaperSimAsNMA}.
declare module Bmlwe <: MLWE_Reduction.
declare module Bbimodal <: BimodalSelfTargetMSIS_Reduction.
```

This listing is representative, not exhaustive. A complete reimplementation must audit every `declaremodule` line in `HAETAE_Security.ec` and in the imported hop-game files. Omitting one restriction can turn a meaningful game proof into a circular proof in which an adversary has access to hidden state or procedures.

7.4 Common failure modes

Overstating the theorem. Do not write "the EasyCrypt proof proves HAETAE EUF-CMA security" without the modifiers "formal model", "random oracle", "conditional", and "fixed ledger".

Counting a loss twice. If a loss is already internalized into a hop premise or bridge lemma, do not add it as a separate final summand.

Erasing the query-bound boundary. The constants $q_H = q_S = 2$ are fixed in the current ledger. They are not a symbolic arbitrary-query theorem.

Confusing SHAKE support with ROM security. `HAETAE_FIPS202.ec` and `HAETAE_Keccak1600.ec` are modeled hash support files. They do not prove that SHAKE instantiates the random oracle.

Using proof holes. If a fact is not proved, make it a premise or an explicitly documented assumption. Do not hide it behind an axiom in the manifest proof surface.

Chapter 8

Minimal Build and Verification Artifacts

8.1 Verification script template

A reimplementation can use this shell skeleton.

```
#!/usr/bin/env sh
set -eu

EASYCRYPT=${EASYCRYPT:-easycrypt}
ROOT=$(pwd)
PROOF_DIR="$ROOT/provable-security/easycrypt"
MANIFEST="$ROOT/provable-security/proof-files.txt"
LOG_DIR="$ROOT/provable-security/logs"

mkdir -p "$LOG_DIR"

failed=0
while IFS= read -r file; do
  case "$file" in
    ""|\#*) continue ;;
  esac

  echo "=== $file ==="
  if "$EASYCRYPT" compile "$PROOF_DIR/$file" \
      -I "$PROOF_DIR" \
      -I "$ROOT/kyber-security" \
      > "$LOG_DIR/$file.log" 2>&1; then
    echo "PASS $file"
  else
    echo "FAIL $file"
    failed=1
  fi
done < "$MANIFEST"

if [ "$failed" -eq 0 ]; then
  echo "RESULT: PASS"
else
```

```
  echo "RESULT: FAIL"
  exit 1
fi
```

8.2 Documentation build

This report can be built from the `report/` directory with:

```
make
```

or directly with:

```
pdflatex -interaction=nonstopmode -halt-on-error \
  haetae-provable-security-reimplementation-guide
pdflatex -interaction=nonstopmode -halt-on-error \
  haetae-provable-security-reimplementation-guide
```

Chapter 9

Final Acceptance Criteria

A reimplementation should not be considered complete until all criteria below hold.

Checkpoint 9.1. The reimplementation is checked against the companion EasyCrypt source files or against an independently expanded document that contains complete source, full lemma statements, proof scripts, and exact module restriction sets. This report alone is not treated as sufficient source material.

Checkpoint 9.2. The report user records which reproduction mode is being claimed. A claim of literal standalone reproduction is rejected unless the documentation has been expanded beyond this roadmap into a complete source-and-proof archive.

Checkpoint 9.3. Every schematic snippet copied from this report has been replaced by checked EasyCrypt source. The active proof surface contains no executable ellipsis, `TODO`, `FIXME`, `admit.`, `sorry.`, or hidden `axiom`.

Checkpoint 9.4. The reimplementation includes a traceability matrix mapping each final-theorem dependency to either an exact companion-source declaration or an independently checked replacement.

Checkpoint 9.5. The manifest-driven EasyCrypt proof gate terminates with `RESULT:PASS`.

Checkpoint 9.6. The declaration inventory has no axioms and no admitted proofs in the active proof surface.

Checkpoint 9.7. The final theorem statement has the same logical shape as `euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal`.

Checkpoint 9.8. The documentation states that the theorem is conditional, fixed-ledger, formal-model, and random-oracle-model.

Checkpoint 9.9. The documentation distinguishes understanding the checked formal-model theorem from understanding or certifying end-to-end security of the HAETAE digital signature algorithm.

Checkpoint 9.10. The documentation gives a clear negative answer to any claim that the report alone suffices both to understand HAETAE algorithm security and to reproduce the machine-checked EasyCrypt verification.

Checkpoint 9.11. Before any completed script-companion claim is made, the documentation is complete enough for mathematicians and cryptologists to understand the mathematical content of the provided `.ec` scripts from the report: declarations, state variables, module restrictions, lemma statements, proof-script blocks, tactic roles, assumptions, and global security use are all explained.

Checkpoint 9.12. Before any completed script-companion claim is made, the script-companion coverage ledger has no open rows. Each manifest file and each security-relevant declaration, module, lemma, proof block, tactic pattern, module restriction, and external assumption is linked to a mathematical explanation in the report.

Checkpoint 9.13. The documentation states that the current final-hop premise is not a non-vacuous concrete-security premise because of `rejection_sampling_loss_term`.

Checkpoint 9.14. Any claim of standalone documentation sufficiency is backed by literal EasyCrypt source, complete proof scripts, exact module restriction sets, full helper-lemma statements, an import/library contract, and a successful verification transcript.

Checkpoint 9.15. The model/specification bridge is recorded as an audited or assumed boundary, not as a checked equivalence theorem.

Checkpoint 9.16. The proof does not claim concrete SHAKE instantiation, implementation correctness, concrete lattice bit security, or reduction runtime bounds.

Chapter 10

Summary

The HAETAE EasyCrypt proof is best reimplemented as a layered proof surface, using this report as the map and the checked EasyCrypt files as the authoritative source. First define parameters, algebra, distributions, events, and assumption games. Then define generic ROM signature games and instantiate them with the HAETAE scheme model. Next expose transcript state, rejection state, random-oracle programming state, and budgeted signing-oracle invariants. Finally assemble the top-level theorem by composing checked hop premises, declared hardness bounds, and the counted-ROM arithmetic ledger.

The central discipline is not merely to make EasyCrypt accept a long file. The theorem must say exactly what has been proved. For the current artifact, that is a conditional probability implication for the formal HAETAE random-oracle model, with a fixed counted-ROM ledger and an explicit vacuity warning on the final rejection-sampling premise. When the document-only question is asked, the answer is no: this report is not, by itself, a proof of deployed algorithm security, a SHAKE-instantiation theorem, or a standalone machine-checkable archive. A faithful reimplementation preserves that boundary while making every program, event, loss, and reduction interface visible to the checker. The report is therefore an entry point for understanding and auditing the proof; the proof itself remains the checked EasyCrypt source plus its manifest-driven verification run.

If the `.ec` scripts are distributed together with this document, the documentation burden increases. The report must then serve as a self-contained mathematical reading guide to those scripts: every security-relevant definition, game, invariant, module restriction, lemma, tactic block, and assumption must have a documented mathematical interpretation. That still does not make the prose a substitute for machine checking, but it should make the scripts intelligible to a mathematician or cryptologist without relying on unwritten project context. The present version records this obligation and supplies a partial guide; it becomes a completed script companion only after the coverage ledger has no open rows.